

Министерство образования и науки
Краснодарского края
ГБПОУ КК ЕПК

Отделение педагогики
Специальность 09.02.07 Информационные системы и программирование

Студент И-42 группы
ДРОБОТОВ МАКСИМ НИКОЛАЕВИЧ

**АВТОМАТИЗИРОВАННАЯ СИСТЕМА УЧЕТА ЛЕКАРСТВЕННЫХ
ПРЕПАРАТОВ**

Дипломная работа (Дипломный проект)

Научный руководитель:
Фомин Андрей Томасович

г. Ейск, 2026 г.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	5
ВВЕДЕНИЕ.....	6
ГЛАВА 1 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	8
1.1 МОДЕЛЬ ПРЕДМЕТНОЙ ОБЛАСТИ.....	8
1.2 ПРЕДВАРИТЕЛЬНОЕ ОПРЕДЕЛЕНИЕ ТРЕБОВАНИЙ.....	10
1.2.1 Общая характеристика системы.....	11
1.2.2 Функциональные возможности.....	12
1.2.3 Работа с данными.....	14
1.2.4 Пользовательский интерфейс.....	16
1.2.5 Атрибуты качества.....	21
1.3 ПОКАЗАТЕЛИ ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ.....	22
1.3.1 Расчет себестоимости программного продукта.....	23
1.3.2 Расчет экономического эффекта программного продукта.....	25
ГЛАВА 2 РЕАЛИЗАЦИЯ ПРОГРАММНЫХ МОДУЛЕЙ.....	27
2.1 ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ.....	27
2.1.1 Методология разработки.....	27
2.1.2 Языки программирования.....	29
2.1.3 Архитектурные подходы.....	30
2.1.4 Инструменты и среды разработки.....	32
2.1.5 Библиотеки и фреймворки.....	33
2.1.6 Тестирование и отладка.....	33
2.2 ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ.....	35
2.2.1 Модуль авторизации и управления сессией.....	36
2.2.2 Модуль продаж.....	36
2.2.3 Модуль справочника лекарств.....	37
2.2.4 Модуль складского учёта.....	38
2.2.5 Модуль управления сотрудниками.....	39

2.2.6 Модуль истории продаж и отчётов.....	39
2.2.7 Модуль базы данных.....	40
2.2.8 Модуль логирования.....	40
2.2.9 Модуль настроек.....	41
2.2.11 Главный модуль.....	43
2.3 ИНТЕГРАЦИЯ ПРОГРАММНЫХ МОДУЛЕЙ.....	45
2.3.1 Механизмы взаимодействия модулей.....	45
2.3.2 Совместное использование ресурсов.....	48
2.3.3 Обработка ошибок при интеграции.....	55
2.4 ОПИСАНИЕ ПРОГРАММНОГО ИНТЕРФЕЙСА.....	61
2.4.1 Окно авторизации.....	62
2.4.2 Окно продаж.....	63
2.4.3 Окно справочника лекарств.....	64
2.4.4 Окно управления складом.....	65
2.4.5 Окно управления сотрудниками.....	66
2.4.6 Окно истории продаж.....	68
2.4.7 Окно настроек.....	69
2.4.8 Обратная связь и сообщения пользователю.....	70
ЗАКЛЮЧЕНИЕ.....	72
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	74
ПРИЛОЖЕНИЕ А.....	75
Листинги программного кода.....	75
ПРИЛОЖЕНИЕ Б.....	78
ТЕСТ-СЬЮТ 1: МОДУЛЬ ПРОДАЖ.....	78
ТЕСТ-СЬЮТ 1: БЕЗОПАСНОСТЬ И АУТЕНТИФИКАЦИЯ.....	82
ТЕСТ-СЬЮТ 2: УПРАВЛЕНИЕ СЕССИЯМИ И ПРАВАМИ ДОСТУПА.....	86
ТЕСТ-СЬЮТ 3: CRUD-ОПЕРАЦИИ И ЦЕЛОСТНОСТЬ ДАННЫХ.....	88
ПРИЛОЖЕНИЕ В.....	92
Документирование программного кода.....	92

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем курсовом проекте применяются следующие сокращения и обозначения.

АРМ — автоматизированное рабочее место

БД — база данных

ИИ — искусственный интеллект

ИКТ — информационно-коммуникационные технологии

ИТ — информационные технологии

ЛВС — локальная вычислительная сеть

МЭК (IEC) — Международная электротехническая комиссия

ОС (OS) — операционная система

ПК (PC) — персональный компьютер

ПО — программное обеспечение

ПЭВМ — персональная электронная вычислительная машина

САПР — система автоматизации проектирования

СУБД (DBMS) — система управления базой данных

СХД — система хранения данных

ЭВМ — электронная вычислительная машина

ВВЕДЕНИЕ

Актуальность исследования.

В современной розничной торговле лекарственными средствами существенно возрастают требования к скорости, точности и прозрачности учёта продаж. Аптечные организации обязаны соблюдать строгие правила хранения, сроки годности, предметно-количественный учёт рецептурных препаратов, а также оперативно формировать отчётность. Ведение учёта вручную или с помощью устаревших информационных систем приводит к ошибкам, потерям данных и снижению качества обслуживания. Поэтому разработка программы для автоматизации учёта продаж лекарственных средств, адаптированной под реальные бизнес-процессы аптеки, является актуальной задачей.

Объект исследования – предметная область «Аптека» как совокупность информационных и управленческих процессов розничной реализации лекарственных препаратов и товаров аптечного ассортимента.

Предмет исследования – процессы автоматизации информационных процессов в предметной области «Аптека», включая приём, регистрацию, учёт продаж и формирование отчётности по движению лекарственных средств.

Цель работы – разработка программного обеспечения для автоматизации учёта продаж лекарственных средств в аптеке.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области «Аптека»: выделить основные сущности, бизнес-процессы и информационные потоки.
2. Изучить существующие аналоги программ для автоматизации аптечного учёта, выявить их преимущества и недостатки применительно к малой и средней аптеке.
3. Сформулировать функциональные и нефункциональные требования к разрабатываемой программе.

4. Выбрать и обосновать средства разработки (язык программирования, СУБД, среду разработки), архитектуру приложения.
5. Спроектировать базу данных для хранения информации о лекарственных средствах, продажах и пользователях системы.
6. Разработать модули учёта продаж, справочников, поиска и генерации отчётов.
7. Выполнить тестирование программного обеспечения и оценить эффективность его внедрения.

Методы исследования:

- анализ нормативной документации по аптечной деятельности (приказы Минздрава, правила предметно-количественного учёта);
- системный анализ бизнес-процессов аптеки;
- методы объектно-ориентированного проектирования (UML);
- моделирование баз данных (ER-диаграммы);
- прототипирование и итеративная разработка ПО.

Практическая значимость работы заключается в создании готового к эксплуатации программного продукта, который может быть использован в реальной аптечной организации для:

- учёта поступления и продажи лекарственных средств;
- контроля остатков и сроков годности;
- автоматического формирования товарных отчётов и чеков продаж;
- снижения временных затрат фармацевтов на рутинные операции.

Структура работы: дипломный проект состоит из введения, двух глав, заключения, списка литературы. Во введении обоснована актуальность темы исследования. В первой главе описаны модель предметной области, требования и показатели экономической эффективности. Вторая глава посвящена описанию процесса разработки программного продукта.

ГЛАВА 1 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

1.1 МОДЕЛЬ ПРЕДМЕТНОЙ ОБЛАСТИ

Аптека является ключевым и высокоспециализированным звеном в системе обеспечения населения лекарственными средствами и изделиями медицинского назначения. Её основная ценность заключается не только в наличии товара, но и в предоставлении квалифицированных фармацевтических услуг, включающих изготовление лекарственных форм и консультирование, что обеспечивает безопасность, эффективность и доступность лекарственной помощи. Эта миссия формирует высокие требования к точности, оперативности и ответственности на всех этапах работы.

Основу деятельности составляют взаимосвязанные бизнес-процессы, образующие непрерывный цикл. Центральными из них являются товарный учёт и управление запасами, приёмка и хранение товара, отпуск лекарственных средств, а также изготовление лекарственных форм по рецептам. Цикл начинается с планирования закупок и работы с поставщиками, что требует тщательного контроля соблюдения нормативов и, что критически важно, сроков годности каждой единицы товара. Затем следует этап приёмки, маркировки и ответственного хранения, который формирует материальную основу для всех последующих операций. Параллельно ведётся непосредственная работа с запросами и рецептами пациентов, включающая их проверку, консультирование и профессиональный подбор лекарств. Кульминацией является процесс отпуска, который подразумевает не просто передачу товара, а финальную проверку дозировок, контроль возможных взаимодействий с другими препаратами и информирование пациента.

Однако эффективность этой сложной цепочки серьёзно страдает из-за большого объёма рутинных операций и сильной зависимости от человеческого

фактора. Зачастую ключевые функции — учёт остатков, контроль сроков годности, поиск препаратов и формирование обязательной отчётности — ведутся вручную или в устаревших, неинтегрированных системах. Это неминуемо ведёт к системным ошибкам: просчётам в заказах поставщикам, возникновению просроченных товарных остатков, дефициту востребованных препаратов и, что наиболее опасно, к рискам при отпуске. Специалисты вынуждены тратить значительную часть рабочего времени на механические операции — подсчёт единиц товара, физический поиск на складе, работу с бумажными журналами и сводками. Следствием становятся очереди, снижение качества консультаций, рост операционных издержек и репутационные риски.

Рассматриваемый бизнес-процесс может быть представлен в виде BPMN-нотации, которая представлена на рисунке 1.

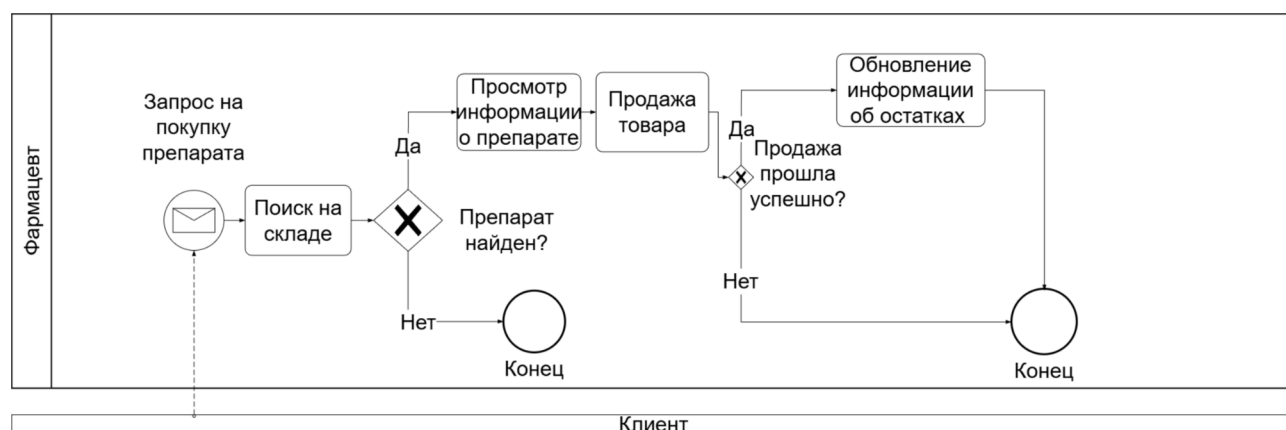


Рисунок 1 — BPMN-нотация бизнес процесса

Таким образом, очевидной и необходимой становится автоматизация, фокус которой должен быть направлен на оптимизацию ключевых операций и минимизацию ручного труда. Речь идёт о внедрении специализированных аппаратно-программных комплексов, которые бы централизовали управление товарными запасами, продажами и нормативными требованиями в едином информационном пространстве. Такая система должна стать цифровым фундаментом, обеспечивающим мгновенное обновление остатков, точный контроль сроков, автоматизацию рутинных расчётов и формирование

регламентированной отчётности. Конечной целью является не просто сокращение издержек, а качественное повышение уровня фармацевтического обслуживания за счёт повышения скорости, точности работы и высвобождения времени персонала для выполнения его ключевой профессиональной задачи — квалифицированного консультирования и обеспечения безопасности пациентов.

После автоматизации рассматриваемого бизнес-процесса его можно будет представить в виде BPMN-нотации, показанной на рисунке 2.

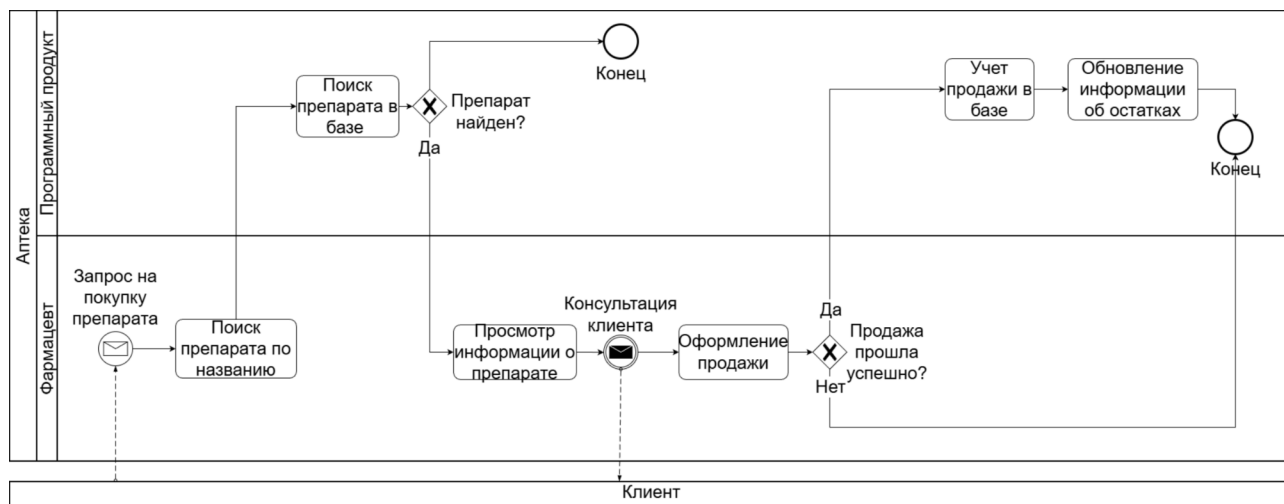


Рисунок 2 — BPMN-нотация бизнес-процесса после автоматизации

1.2 ПРЕДВАРИТЕЛЬНОЕ ОПРЕДЕЛЕНИЕ ТРЕБОВАНИЙ

Формирование перечня функциональных требований является обязательным этапом, предшествующим разработке программного продукта. Чёткая спецификация позволяет определить границы системы и избежать разночтений между заказчиком и разработчиком. Для удобства отслеживания и верификации каждое требование получает уникальный идентификатор в формате:

<префикс>-<номер> <текст требования>

В качестве префиксов использовались следующие сокращения:

REQ — общие требования

FUNC — функциональные требования
NONF — нефункциональные требования
SEC — безопасность
PERF — производительность
UI — интерфейс пользователя
DATA — данные
COM — коммуникации
HW — требования к интерфейсам оборудования
QUAL — атрибут качества

1.2.1 Общая характеристика системы

REQ-001 Система должна представлять собой настольное кроссплатформенное приложение (Windows/Linux/macOS), создающее единую информационную среду для управления товарным учетом, продажами и персоналом аптеки.

REQ-002 Система должна функционировать со встроенной локальной базой данных SQLite без необходимости установки и настройки отдельного сервера баз данных.

REQ-003 Система должна поддерживать работу в режиме реального времени: изменения остатков при продаже должны фиксироваться мгновенно (в рамках транзакции БД).

REQ-004 Система должна реализовывать многооконный интерфейс с использованием QStackedWidget для переключения между экранами.

REQ-005 Система должна обеспечивать разграничение доступа между ролями «Администратор» и «Провизор» с соответствующим набором функциональных прав.

1.2.2 Функциональные возможности

FUNC-001 Система должна предоставлять форму входа с полями «Логин» и «Пароль».

FUNC-002 Система должна выполнять хеширование паролей по алгоритму SHA-256 перед записью в базу данных и при проверке.

FUNC-003 При успешной авторизации система обязана установить сессию пользователя (ID, имя, роль).

FUNC-004 При неудачной авторизации система должна выводить сообщение: «Пользователь не найден» или «Неверный пароль».

FUNC-005 Система должна обеспечивать поиск лекарств в реальном времени по названию и МНН с учетом вводимых символов.

FUNC-006 Пользователь должен иметь возможность добавлять выбранное лекарство в корзину двойным щелчком мыши или нажатием кнопки «Добавить в корзину».

FUNC-007 Система должна отображать корзину в виде таблицы с колонками: Название, Цена, Количество, Сумма.

FUNC-008 Пользователь должен иметь возможность изменять количество товара в корзине с помощью элемента SpinBox.

FUNC-009 Система должна автоматически пересчитывать общую сумму корзины и итоговую стоимость при изменении количества любого товара.

FUNC-010 При оформлении продажи система обязана выполнить все операции (создание продажи, добавление позиций, списание товара) в единой транзакции базы данных.

FUNC-011 Если в процессе оформления продажи возникает ошибка (недостаток товара, сбой БД), система должна выполнить полный откат транзакции.

FUNC-012 Система должна генерировать уникальный номер чека в формате «SALE-ГГГГММДДЧЧММСС».

FUNC-013 Система должна отображать чек продажи в модальном диалоговом окне после успешного завершения операции.

FUNC-014 Система должна автоматически сохранять текстовую копию чека в папку «receipts/» с именем «receipt_<номер_чека>.txt».

FUNC-015 Пользователь должен иметь возможность выбора способа оплаты из справочника (наличные, карта).

FUNC-016 Система должна предоставлять таблицу со списком всех лекарств с колонками: ID, название, МНН, форма выпуска, производитель, категория, цена продажи, остаток.

FUNC-017 Пользователь должен иметь возможность фильтрации списка лекарств по категории и лекарственной форме с помощью выпадающих списков.

FUNC-018 Администратор должен иметь возможность добавлять новое лекарство, редактировать существующее и удалять лекарство (при отсутствии ссылок в продажах).

FUNC-019 Система должна выводить предупреждение при попытке удалить лекарство, которое используется в продажах (нарушение внешнего ключа).

FUNC-020 Система должна предоставлять таблицу управления складом с колонками: ID, лекарство, партия, срок годности, количество, цена закупки, цена продажи, дата поступления.

FUNC-021 Администратор должен иметь возможность добавлять новую партию товара на склад с указанием лекарства, номера партии, срока годности, количества, цен.

FUNC-022 Администратор должен иметь возможность редактировать и удалять записи склада (с проверкой использования в продажах).

FUNC-023 Система должна обеспечивать фильтрацию складских записей по категории и конкретному лекарству.

FUNC-024 Система должна предоставлять историю продаж за выбранный пользователем период (по умолчанию — последние 30 дней).

FUNC-025 При выборе продажи из списка система должна отображать состав этой продажи (название, количество, цена, сумма).

FUNC-026 Система должна формировать отчеты по складу в форматах HTML, TXT, JSON и CSV.

FUNC-027 Система должна формировать отчеты по продажам в форматах HTML, TXT и JSON.

FUNC-028 Система должна генерировать отчеты в отдельном потоке, чтобы не блокировать пользовательский интерфейс.

FUNC-029 Система должна открывать созданный отчет в программе по умолчанию, если соответствующая опция включена в настройках.

FUNC-030 Администратор должен иметь возможность добавлять, редактировать и удалять сотрудников.

FUNC-031 Система должна позволять администратору изменять пароль сотрудника без необходимости ввода старого пароля.

1.2.3 Работа с данными

DATA-001 Система должна хранить данные о лекарствах, включая: ID, название, международное непатентованное название (МНН), ID формы выпуска, ID производителя, ID категории, флаг активности.

DATA-002 Система должна хранить данные о складских партиях: ID, ID лекарства, номер партии, срок годности, количество, цена закупки, цена продажи, дата поступления.

DATA-003 Система должна хранить данные о продажах: ID, номер чека, дата продажи, ID сотрудника, итоговая сумма, ID способа оплаты.

DATA-004 Система должна хранить состав продажи: ID, ID продажи, ID складской позиции, количество, цена продажи на момент продажи.

DATA-005 Система должна хранить данные о сотрудниках: ID, ФИО, логин, хеш пароля (SHA-256), ID должности.

DATA-006 Система должна хранить справочники: должности, категории лекарств, лекарственные формы, производители, способы оплаты.

DATA-007 Система должна обеспечивать целостность данных на уровне внешних ключей: продажи не могут существовать без сотрудника, позиции чека — без продажи, складские записи — без лекарства.

DATA-008 При удалении лекарства, связанного со складскими записями или продажами, система должна отклонять операцию с сообщением о внешнем ключе.

DATA-009 Система должна хранить настройки приложения с помощью QSettings в реестре Windows или конфигурационных файлах.

DATA-010 Система должна сохранять логи действий и ошибок в текстовые файлы с возможностью ротации при превышении максимального размера.

DATA-011 Лог-файлы должны содержать временную метку, уровень сообщения (DEBUG, INFO, WARNING, ERROR, CRITICAL), имя файла и строку исходного кода.

DATA-012 База данных для разрабатываемого продукта должна соответствовать ER-диаграмме, показанной на рисунке 3

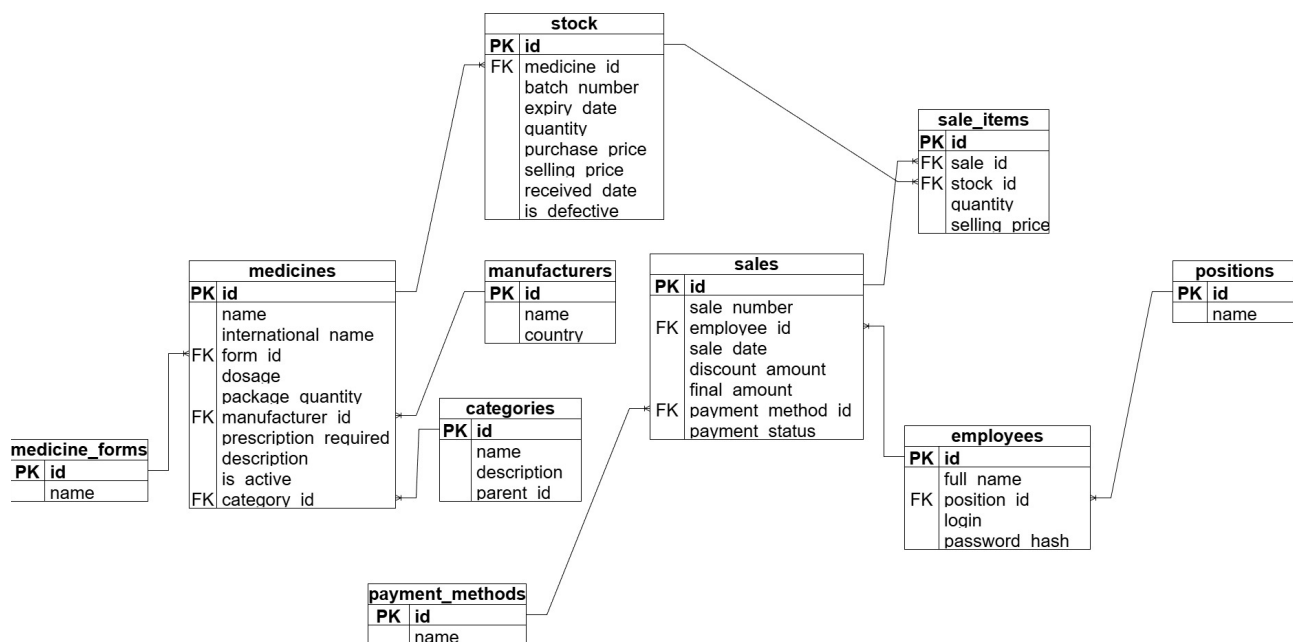


Рисунок 3 — ER-диаграмма базы данных

1.2.4 Пользовательский интерфейс

1.2.4.1 Пользовательские интерфейсы

Интерфейсы окон приложения должны соответствовать эскизам, показанным на рисунках 4-9.

Файл О программе

Вход в систему

Логин

Пароль

Войти

Рисунок 4 — Эскиз окна авторизации

Файл О программе

Список лекарств

Поиск лекарств

Поиск лекарств.....

Выход

Лекарства

Добавить в корзину

Корзина:

Корзина

Удалить из корзины

Способ оплаты

Оплатить

ИТОГ

Рисунок 5 — Эскиз окна продаж

Файл О программе	
Поиск	
Таблица "Сотрудники"	
Добавить Ред. Удалить Обновить	Назад

Рисунок 6 — Эскиз таблицы просмотра сотрудников

Файл О программе	
Выбор даты Обновить Отчет	
Продажи:	
Таблица продажи	
Состав продажи:	
Состав продажи	

Рисунок 7 — Эскиз таблицы просмотра истории продаж

Файл О программе		
Назад		
Поиск	Категория	Лекарства
Таблица "Склад"		
Добавить	Ред.	Удалить Отчет Обновить

Рисунок 8 — Эскиз окна просмотра склада

Файл О программе		
Назад		
Поиск		
Категория	Форма	
Таблица лекарств		
Добавить Редактировать Удалить		

Рисунок 9 — Эскиз окна просмотра лекарств

1.2.4.2 Интерфейсы программного обеспечения

COM-001 Система должна функционировать под управлением операционных систем Windows 10/11, Linux и macOS 11+.

COM-002 Система должна использовать Qt 5.15 или новее в качестве фреймворка для графического интерфейса.

COM-003 Система должна взаимодействовать с SQLite версии 3.x через драйвер QSQLITE.

COM-004 Система не должна зависеть от внешних сетевых сервисов или облачных решений.

COM-005 Для создания PDF-отчетов система должна формировать HTML-файлы с последующим просмотром через браузер (функция прямой печати PDF не реализована).

COM-006 Система должна использовать QSettings для кроссплатформенного хранения настроек (реестр Windows, property list macOS, INI-файлы Linux).

1.2.4.3 Интерфейсы оборудования

HW-001 Минимальный объем оперативной памяти: 512 МБ.

HW-002 Минимальный объем свободного дискового пространства: 200 МБ (без учета БД и логов).

HW-003 Процессор: архитектура x86 или x64 с тактовой частотой от 1 ГГц.

HW-004 Разрешение экрана: не менее 1024×768 пикселей (рекомендуется 1280×720).

HW-005 Система не требует специализированного оборудования (фискальные регистраторы, принтеры чеков не поддерживаются).

HW-006 Для печати отчетов и чеков в формате PDF система использует программный драйвер виртуального принтера или встроенную функцию сохранения.

1.2.4.4 Коммуникационные интерфейсы

NET-001 Система не требует сетевого подключения для базового функционирования (автономное рабочее место).

NET-002 При сохранении отчетов и чеков система работает только с локальной файловой системой, сетевые протоколы не используются.

NET-003 Система не реализует многопользовательский режим работы с общей базой данных (только однопользовательский локальный доступ).

1.2.5 Атрибуты качества

QUAL-001 Все таблицы должны поддерживать сортировку по любому столбцу при клике на заголовок.

QUAL-002 Система должна отображать строку состояния с информацией о текущем действии и приветствием пользователя.

QUAL-003 Все диалоговые окна (добавление/редактирование) должны иметь кнопки «Сохранить» и «Отмена».

QUAL-004 Чек после продажи должен отображаться в моноширинном форматировании для удобства чтения.

QUAL-005 Поиск лекарств должен откликаться на ввод пользователя с задержкой не более 300 мс без зависания интерфейса.

QUAL-006 Генерация отчетов должна выполняться в фоновом потоке, интерфейс должен оставаться отзывчивым.

QUAL-007 Загрузка списка лекарств (до 10 000 записей) должна выполняться не более 2 секунд.

QUAL-008 Пароли не должны храниться в базе данных в открытом виде (только хеш SHA-256).

QUAL-009 Неавторизованный пользователь не должен иметь доступа ни к одному из функциональных окон системы.

QUAL-010 Пользователь с ролью «Провизор» не должен видеть кнопки «Добавить», «Редактировать», «Удалить» в окнах лекарств, склада и сотрудников.

QUAL-011 Пользователь с ролью «Провизор» не должен иметь доступа к пунктам меню «Сотрудники», «Продажи» (история) и «Склад».

QUAL-012 Система не должна выполнять автоматических изменений файловой системы без явного действия пользователя (кроме логов и сохранения чеков).

QUAL-013 Перед удалением записи система должна запрашивать подтверждение у пользователя.

QUAL-014 При попытке удалить запись, связанную внешними ключами с другими таблицами, система должна выводить понятное пользователю сообщение о невозможности удаления с указанием причины.

QUAL-015 Система должна логировать критические ошибки и попытки несанкционированного доступа (неверный пароль).

1.3 ПОКАЗАТЕЛИ ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ

Внедрение разрабатываемого программного продукта в деятельность аптечной организации направлено на автоматизацию ключевых бизнес-процессов, что, согласно сформулированным требованиям, должно привести к снижению трудоемкости операций, оптимизации товарных запасов и повышению качества обслуживания. Для обоснования коммерческой целесообразности разработки и внедрения системы необходимо произвести расчет экономической эффективности проекта. [1]

1.3.1 Расчет себестоимости программного продукта

Расчет себестоимости программного продукта производится по формуле:

$$C = M_{\text{вс}} + \mathcal{E} + Z_{\text{зн}}^{\text{о}} + Z_{\text{зн}}^{\text{д}} + Z_{\text{сн}} + H, \quad (1)$$

где $M_{\text{вс}}$ — затраты на вспомогательные материалы, Р .

$\mathcal{E}$ — затраты на электроэнергию и технологические цели, Р

$Z_{\text{зн}}^{\text{о}}$ — основная зарплата работника, Р

$Z_{\text{зн}}^{\text{д}}$ — дополнительная зарплата разработчика, Р

$Z_{\text{сн}}$ — отчисления на соц. нужды, Р

H — накладные расходы, Р

Таблица 1 — Затраты на вспомогательные материалы

	Наименование затрат	Кол-во, шт.	Сумма, Р .
1.	Картридж для принтера HP LJ	1	1200
2.	Упаковка бумаги	1	350
3.	Память USB flash drive, 128 ГБ	1	1400
	Итого	2950	

Затраты на электроэнергию рассчитываются по формуле 2:

$$\mathcal{E} = P \cdot C_{\text{э}} \cdot T_{\text{общ}} \cdot R_{\text{заг}}, \quad (2)$$

где

P — мощность потребляемой энергии, кВт

$C_{\text{э}}$ — стоимость одного киловатт-часа электроэнергии, р./кВт-ч

$T_{\text{общ}}$ — общие затраты труда на разработку ПО

$R_{\text{заг}}$ — коэффициент загрузки компьютера

Основная заработная плата разработчика рассчитывается по формуле 3:

$$Z_{\text{зн}}^{\text{о}} = C_{\text{чтс}} \cdot T_{\text{общ}}, \quad (3)$$

где

$C_{\text{чтс}}$ — часовая тарифная ставка разработчика, р./час ;

$T_{\text{общ}}$ — общие затраты труда на разработку ПО

Дополнительная з/плата разработчика составляет 10% от основной заработной платы и рассчитывается по формуле 4:

$$Z_{\text{зп}}^{\text{д}} = 0.1 \cdot Z_{\text{зп}}^{\text{о}} \quad (4)$$

Отчисления на соц. нужды определяются по формуле 5:

$$Z_{\text{сн}} = (Z_{\text{зп}}^{\text{д}} + Z_{\text{зп}}^{\text{о}}) \cdot R_{\text{сн}} \quad (5)$$

Накладные расходы рассчитываются по формуле 6:

$$H = 0,1 \cdot (Z_{\text{зп}}^{\text{д}} + Z_{\text{зп}}^{\text{о}}) \quad (6)$$

Зная все числовые значения, можно рассчитать полную себестоимость разрабатываемого программного продукта.

Затраты на электроэнергию и технологические цели:

$$\Xi = 0,25 \cdot 10,9 \cdot 800 \cdot 0,6 = 1\,308 \text{ р.}$$

Основная зарплата работника:

$$Z_{\text{зп}}^{\text{о}} = 550 \cdot 800 = 440\,000 \text{ р.}$$

Дополнительная зарплата разработчика:

$$Z_{\text{зп}}^{\text{д}} = 0.1 \cdot 440\,000 = 44\,000 \text{ р.}$$

Отчисления на соц. нужды:

$$Z_{\text{сн}} = (440\,000 + 44\,000) \cdot 0,30 = 145\,200 \text{ р.}$$

Накладные расходы:

$$H = 0,1 \cdot (440\,000 + 44\,000) = 48\,400 \text{ р.}$$

Таблица 2 — Калькуляция себестоимости программного продукта

	Наименование статей расходов	Затраты, Р
1.	Основная з/плата разработчика	440 000
2.	Дополнительная заработная плата разраб.	44 000
3.	Отчисления на соц. нужды	145 200
4.	Накладные расходы	48 400
5.	Затраты на эл.энергию	1 308
6.	Вспомогательные материалы	2 950
	Полная себестоимость продукта	681 858

1.3.2 Расчет экономического эффекта программного продукта

Экономический эффект от внедрения ПО рассчитывается по формуле 7:

$$\mathcal{E}_{\text{эф}} = N \cdot (t_1 - t_2) \cdot C_{\text{чтс}} \cdot K_{\text{доп}} \cdot K_{\text{сн}} - E_{\text{н}} \cdot K, \quad (7)$$

где

N — количество обрабатываемых документов;

t_1, t_2 — трудоемкость обработки документов до и после внедрения программы:

$$t_1 = 15, \quad t_2 = 7$$

$C_{\text{чтс}}$ — часовая тарифная ставка пользователя, р./час;

$K_{\text{доп}}$ — коэффициент отчислений на дополнительную заработную плату, $K_{\text{доп}} = 0,1$;

$K_{\text{сн}}$ — коэффициент отчислений на соц. нужды, $K_{\text{сн}} = 0,3$;

$E_{\text{н}}$ — нормативный коэффициенткупаемости капитальных вложений, $E_{\text{н}} = 0,39$;

K — доп. капиталовложения, связанные с разработкой программного продукта.

Срок окупаемости программного продукта рассчитывается по формуле:

$$T_{\text{ок}} = \frac{C_{\text{пр}}}{C_1 - C_2}$$

где $C_{\text{пр}}$ — себестоимость программного продукта, ₴

C_1 — стоимостные затраты при использовании ручного труда, ₴

C_2 — стоимостные затраты при использовании программы, ₴

Для расчета стоимостных затрат нужно взять формулы 8 и 9.

$$\mathcal{E}_1 = N \cdot t_1 \cdot C_{\text{чтс}} \cdot K_{\text{доп}} \cdot K_{\text{сн}} - E_{\text{н}} \cdot K \quad (8)$$

$$\mathcal{E}_2 = N \cdot t_2 \cdot C_{\text{чтс}} \cdot K_{\text{доп}} \cdot K_{\text{сн}} - E_{\text{н}} \cdot K \quad (9)$$

Годовая экономия эксплуатационных затрат ($C_1 - C_2$):

$$(C1 - C2) = 798 \cdot 500 \cdot 1.1 \cdot 1.3 = 570\,570 \text{ р.}$$

Экономический эффект($\Theta_{\text{эф}}$):

$$\Theta_{\text{эф}} = 570\,570 \text{ Р} - 265\,925 \text{ Р} = 304\,645 \text{ р.}$$

Расчёт срока окупаемости ($T_{\text{ок}}$):

$$T_{\text{ок}} = 681\,858 \text{ р.} / 570\,570 \text{ р/год} \approx 1.195 \text{ года}$$

ГЛАВА 2 РЕАЛИЗАЦИЯ ПРОГРАММНЫХ МОДУЛЕЙ

2.1 ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ

2.1.1 Методология разработки

Проектирование программного продукта, предназначенного для ответственной сферы фармацевтического учёта, требовало не только детального анализа предметной области, но и применения структурированной, обоснованной методологии управления процессом создания. Это необходимо для обеспечения управляемости проекта, системного контроля качества и достижения поставленных целей в рамках дипломной работы. Методология определяет, как требования, описанные ранее, будут трансформироваться в конкретный программный код и рабочие функции системы.

Исходя из специфики проекта как индивидуальной работы, определяющими критериями выбора методологии стали: гибкость и адаптивность к возможным изменениям в техническом задании по мере углубления в предметную область и прояснения деталей; эффективность управления задачами без избыточных административных накладных расходов, характерных для крупных команд; а также необходимость регулярного получения и демонстрации работоспособных промежуточных результатов, что позволяет контролировать прогресс и корректировать направление разработки.

Исходя из вышеперечисленных критериев, рассматривались классические и современные варианты. Водопадная модель (Waterfall) с её строгой последовательностью этапов была отвергнута из-за недостаточной гибкости и высокого риска несоответствия финального продукта реальным потребностям, которые могли окончательно сформироваться только в процессе разработки. Формализованные гибкие методологии (Agile), такие как Scrum, с их

фиксированными спринтами, ролями и регулярными ритуалами, создавали излишние организационные накладные расходы для разработки. В результате проведённого сравнительного анализа для успешной реализации данного проекта была признана наиболее целесообразной адаптация упрощённого Agile-подхода на основе принципов Kanban.

Причины выбора заключаются в его прагматичных преимуществах для индивидуальной научно-исследовательской работы. Во-первых, визуализация потока работ через Kanban-доску позволяет наглядно видеть все задачи, их приоритет и текущий прогресс, перемещая их по колонкам «К выполнению», «В работе», «На проверке» и «Выполнено». Это критически важно для самоорганизации и поддержания фокуса. Во-вторых, гибкость и непрерывность процесса, отсутствие жёстких временных рамок спринтов позволяют оперативно вносить изменения в план работ, переключаться между задачами по мере поступления новых требований от научного руководителя или выявления ошибок в ходе тестирования. В-третьих, фокус на ограничении количества одновременно выполняемых задач, выраженный принципом «Stop Starting, Start Finishing», помогает концентрироваться на полном завершении текущей функциональности перед началом новой, что повышает общую эффективность и качество кода. Наконец, минимальные накладные расходы методологии, не требующей проведения формальных совещаний и ведения объёмной промежуточной отчётности, идеально подходят для индивидуального разработчика, позволяя максимум времени уделять непосредственно программированию и решению проектных задач.

План мероприятий по применению выбранной методологии включал создание и постоянное ведение бэклога продукта в виде упорядоченного списка функций и технических задач; визуализацию всего процесса разработки на цифровой Kanban-доске в Trello; а также регулярный пересмотр и актуализацию этого списка по мере углубления в предметную область и получения обратной связи. Рассмотрев возможные альтернативы, было принято решение

использовать Kanban как наиболее практичный и эффективный подход для условий индивидуальной разработки в рамках дипломного проекта, обеспечивающий необходимую гибкость, прозрачность и контроль на всех этапах.

2.1.2 Языки программирования

Успешная реализация методологии разработки напрямую зависит от выбора адекватного технологического стека, который позволит воплотить спроектированные функции в эффективный и надёжный программный код. Основным языком программирования, выбранный для реализации проекта, — C++. Этот выбор является не случайным, а стратегическим, основанным на анализе требований к будущей системе.

Выбор обусловлен несколькими ключевыми факторами, напрямую вытекающими из специфики предметной области. Во-первых, высокая производительность является критическим требованием для системы, которая должна мгновенно обрабатывать запросы к базе данных лекарственных средств, осуществлять сложный поиск по различным критериям, формировать отчёты и производить расчёты в реальном времени, даже при работе с большими объёмами данных на стандартном аптечном компьютере. C++, как компилируемый язык системного уровня, обеспечивает быструю работу приложения и эффективное управление ресурсами. Во-вторых, прямое и детализированное управление памятью и аппаратными ресурсами позволяет создавать оптимизированные и отзывчивые десктопные приложения, что соответствует требованию об удобстве использования и отсутствии задержек при работе пользователя. В-третьих, решающим аргументом стала мощная экосистема фреймворка Qt. Qt, написанный на C++, предоставляет для этого языка идиоматический и удобный API, максимально раскрывая его преимущества при создании графического интерфейса, работе с базами данных и организации логики приложения. Это обеспечивает синергию между

производительностью низкоуровневого языка и продуктивностью высокоуровневого фреймворка. Дополнительно для формирования сложных печатных форм отчётов планируется использовать встроенный механизм шаблонов Qt, а для хранения конфигурации приложения — легковесный и широко поддерживаемый формат JSON, парсинг которого предоставляется библиотекой Qt Core.

2.1.3 Архитектурные подходы

Фреймворк Qt6, выбранный в качестве технологической основы проекта, не просто предоставляет набор виджетов для создания пользовательского интерфейса, но и диктует определенную архитектурную модель построения приложений. В его основе лежат классические паттерны проектирования, мастерски адаптированные для нужд быстрой и эффективной разработки. Далее описываются основные паттерны, использованные при разработке приложения.

2.1.3.1 Singleton

Singleton применяется для классов, которые должны существовать в единственном экземпляре на всё приложение. В приложении этот паттерн применяется для управления тремя ключевыми глобальными сервисами приложения:

- Database - управляет единственным подключением к базе данных SQLite;
- Logger - Обеспечивает централизованное логирование действий и ошибок со всего приложения.
- UserSession - Хранит данные о текущем авторизованном пользователе

2.1.3.2 Model-View

Это основной архитектурный паттерн для отображения всех табличных данных. Он разделяет данные, их представление и логику взаимодействия.

Архитектура Model-View лежит в основе всех окон, работающих с таблицами базы данных: SalesHistoryWindow, MedicinesWindow, StockWindow, EmployeesWindow.

2.1.3.3 Observer/Signal-Slot

Паттерн Signal-Slot является фундаментом для всей коммуникации в проекте, обеспечивая асинхронное и слабосвязанное взаимодействие между UI-элементами, окнами и фоновыми потоками.

2.1.3.4 Strategy

Паттерн Strategy применён для инкапсуляции различных алгоритмов создания отчетов. Выбор конкретной стратегии (формата файла) происходит во время выполнения. Например, метод generateSalesReport() получает желаемый формат в виде строки. В зависимости от значения, он выбирает и вызывает соответствующую стратегию генерации.

2.1.3.5 Active Object

Паттерн Active Object реализован с помощью низкоуровневых классов Qt для выноса блокирующих операций в фоновый поток, что предотвращает зависание интерфейса.

Данный паттерн реализован в SalesHistoryWindow и StockWindow для генерации отчетов.

2.1.3.6 Composite

Паттерн Composite реализован на уровне построения пользовательского интерфейса средствами самого фреймворка Qt. Все экранные формы приложения организованы в виде иерархического дерева виджетов: главное окно Widget содержит QStackedWidget, который, в свою очередь, агрегирует шесть функциональных модулей. Каждый модуль построен по тому же

принципу — контейнеры-лейауты объединяют атомарные элементы управления в единую композицию.

2.1.4 Инструменты и среды разработки

Для практической реализации выбранных архитектурных решений и написания кода на C++ с использованием Qt необходим специализированный инструментарий, повышающий продуктивность и обеспечивающий контроль качества. Процесс разработки проходил с использованием интегрированной среды разработки (IDE) Qt Creator — инструмента, оптимально подходящего для работы с выбранным технологическим стеком.

Выбор Qt Creator обусловлен его глубокой и бесшовной интеграцией с фреймворком Qt. Это включает встроенный визуальный дизайнер форм для быстрого прототипирования интерфейса, мощную систему автодополнения кода, «понимающую» синтаксис и мета-объектную систему Qt (Qt-aware code model), а также упрощённые и гибкие механизмы управления сборкой проекта через qmake или CMake. Кроме того, среда содержит все необходимые профессиональные инструменты: отладчик для пошагового анализа выполнения программы и инспекции переменных, профайлер для поиска узких мест производительности и встроенную систему управления версиями. Именно для контроля версий использовалась распределённая система Git с размещением основного репозитория на платформе GitLab. Это обеспечивает надёжное хранение полной истории изменений исходного кода, возможность создания веток для экспериментов или реализации отдельных функций, простой откат в случае ошибок, а также служит централизованной резервной копией всего проекта, что соответствует принципам надёжности.

2.1.5 Библиотеки и фреймворки

Для ускорения разработки, обеспечения надёжности и реализации сложной функциональности были использованы проверенные временем библиотеки и фреймворки. Ключевым из них является Qt Framework 6.9.1, который предоставил все необходимые компоненты для создания законченного приложения: Qt Widgets для создания классического десктопного графического интерфейса пользователя; Qt SQL для унифицированной абстракции работы с реляционными базами данных, позволяющей легко переключаться между разными СУБД; модуль Qt Core, содержащий базовые классы для работы с контейнерами, строками, файловой системой, парсинга JSON, а также реализующий мощную модель сигналов и слотов для коммуникации между объектами. В качестве системы управления базами данных была выбрана СУБД SQLite.

Выбор перечисленных инструментов обоснован следующими причинами. Высокая зрелость и надёжность Qt и SQLite, подтверждённые многолетним использованием в коммерческих и критически важных проектах, гарантируют предсказуемое поведение и стабильность. Их сочетание идеально соответствует поставленным требованиям: оно позволяет создавать кроссплатформенные десктопные приложения, работающие с локальными данными без необходимости развёртывания отдельного сервера баз данных, что упрощает внедрение в условиях аптеки. Наконец, наличие исчерпывающей официальной документации и активного сообщества пользователей и разработчиков гарантирует возможность быстрого поиска решений при возникновении проблем или необходимости реализации нестандартной функциональности.

2.1.6 Тестирование и отладка

В процессе создания программного продукта разрабатываемые модули активно тестировались для предотвращения возникновения ошибок на этапе

эксплуатации. Особое внимание уделялось компонентам, неисправность которых могла бы напрямую повлиять на достоверность данных и безопасность работы персонала.

Написание модульных тестов (Unit Tests) было сосредоточено на критически важных элементах бизнес-логики. В частности, тестированию подверглись алгоритмы аутентификации и авторизации (включая разграничение ролей «Администратор» и «Сотрудник»), механизмы валидации вводимых данных (проверка уникальности логина, соблюдение ограничений целостности, верификация формата хешированных паролей), а также корректность выполнения базовых операций с основными сущностями системы.

Параллельно проводилось функциональное тестирование пользовательского интерфейса для проверки корректности взаимодействия между элементами управления, их реакции на действия пользователя и синхронизации с underlying-моделями данных.

Далее приведены результаты выполнения трёх тест-сюитов, охватывающих ключевые модули системы: безопасность и аутентификацию, управление сессиями и правами доступа, а также CRUD-операции и целостность данных. Каждый тест-сюит содержит набор тест-кейсов, сгруппированных по функциональному признаку и привязанных к соответствующим пунктам спецификации требований.

Таблица 1 — Результаты выполнения тест-сюитов.

Тест-сюит	Проверяемые требования	Всего тестов	Пройдено	Провалено
TS-SALE-001	Целостность транзакций продаж	7	7	0
TS-SEC-001	Безопасность и аутентификация	5	5	0
TS-SESS-001	Сессии и права доступа	2	2	0
TS-DATA-001	CRUD и целостность данных	5	5	0
Итого		12	12	0

Все тест-сьюты приведены в приложении Б.

По итогам модульного тестирования можно сделать вывод о том, что разработанный программный продукт полностью соответствует поставленным функциональным и нефункциональным требованиям. Все заявленные критические модули - механизмы аутентификации и авторизации, валидация и безопасное хранение данных, CRUD-операции с основными сущностями, а также ограничения целостности базы данных - отработали корректно в изолированной тестовой среде.

Ниже представлен консольный вывод, подтверждающий успешное прохождение модульных тестов для модуля продаж. Все тест-кейсы, проверяющие создание продажи с уникальным номером чека, корректность сохранения итоговой суммы и способа оплаты, добавление позиций в состав чека, атомарное списание остатков со склада, а также невозможность продажи при недостатке товара, выполнены без ошибок.

```
***** Start testing of SalesTest *****
Config: Using QTest library 6.9.1, Qt 6.9.1 (x86_64-little_endian-llp64
shared (dynamic) release build; by GCC 13.1.0), windows 10
PASS   : SalesTest::initTestCase()
QDEBUG : SalesTest::test_sale_created() БД подключена
PASS   : SalesTest::test_sale_created()
QDEBUG : SalesTest::test_sale_amount() БД подключена
PASS   : SalesTest::test_sale_amount()
QDEBUG : SalesTest::test_sale_item_added() БД подключена
PASS   : SalesTest::test_sale_item_added()
QDEBUG : SalesTest::test_stock_decreases_after_sale() БД подключена
PASS   : SalesTest::test_stock_decreases_after_sale()
QDEBUG : SalesTest::test_cannot_sell_more_than_stock() БД подключена
PASS   : SalesTest::test_cannot_sell_more_than_stock()
QDEBUG : SalesTest::test_payment_method_saved() БД подключена
PASS   : SalesTest::test_payment_method_saved()
QDEBUG : SalesTest::test_multiple_items_in_sale() БД подключена
```

```
PASS    : SalesTest::test_multiple_items_in_sale()
PASS    : SalesTest::cleanupTestCase()
Totals: 9 passed, 0 failed, 0 skipped, 0 blacklisted, 14ms
***** Finished testing of SalesTest *****
```

Таким образом, тестирование подтвердило стабильность и надёжность базовых компонентов системы, их соответствие спецификации требований.

2.2 ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ

Разрабатываемый продукт состоит из отдельных функционально независимых модулей, что позволяет локализовать изменения, упрощает тестирование компонентов и повышает общую надёжность приложения. Для успешной реализации и последующего сопровождения программного продукта необходимо чётко понимать, каким образом модули взаимодействуют друг с другом и какие функции выполняет каждый из них.

2.2.1 Модуль авторизации и управления сессией

Модуль авторизации (Login и UserSession) и управления сессией предназначен для аутентификации пользователей в системе, проверки учётных данных и поддержки активной сессии.

Этот модуль состоит из класса Login, который представляет собой форму входа в систему, и класса UserSession – синглтона, хранящего информацию об имени пользователя и его роли.

Основными методами модуля являются:

- Login::on_logBtn_clicked() – обрабатывает нажатие кнопки входа и вызывает проверку через Database::loginUser();
- UserSession::setUser() / getUserId() / getUserName() / getUserRole() - получает id, имя и роль пользователя;

- `UserSession::isAdmin()` – проверяет, является ли пользователь администратором.

Успешный вход генерирует сигнал `userLoggedIn(userId, userName, userRole)` после чего сессия сохраняется в `UserSession`.

Модуль взаимодействует с модулем базы данных для проверки хеша пароля и получения информации о сотруднике. После успешной аутентификации уведомляет главный модуль для переключения на окно продаж.

2.2.2 Модуль продаж

Данный модуль (`SalesWindow`) реализует основную бизнес-функцию – оформление продажи лекарственных средств. Включает поиск товаров, формирование корзины, расчёт суммы, выбор способа оплаты, проведение транзакции и печать/сохранение чека.

Модуль продаж включает в себя таблицу доступных лекарств, таблицу корзины, поля для поиска и фильтров, кнопки добавления и удаления препаратов из корзины, а также кнопку оплаты.

Основными методами модуля являются:

- `searchMedicines()` – осуществляет поиск по названию или международному наименованию препарата;
- `addToCart()` / `addFromMedicines()` – производит добавление товара в корзину;
- `updateCartTotal()` – производит пересчёт итоговой суммы собранной корзины;
- `checkout()` – реализует оформление продажи. При оплате происходит транзакционное обновление базы данных, производится списание остатков, запись продажи и позиций;
- `generateReceipt()` / `saveReceiptToFile()` / `printReceipt()` – формирует и сохраняет чек в формате PDF или TXT.

Модуль получает данные о препаратах из базы данных, после оплаты производятся записи в таблицах sales и sale_items, обновляются данные в таблице stock, формируется чек и генерируется сигнал saleCompleted.

Использует модуль базы данных для выполнения SQL-запросов, UserSession для получения ID сотрудника, модуль для логирования продаж. При необходимости поиска открывает MedicinesWindow.

2.2.3 Модуль справочника лекарств

Данный модуль (MedicinesWindow, MedicineEditDialog) предназначен для просмотра, поиска, добавления, редактирования и удаления записей о лекарственных средствах. Поддерживает фильтрацию по категории и форме выпуска.

Модуль состоит из главного окна для просмотра списка лекарств и диалога MedicineEditDialog, который предназначен для добавления новых лекарственных препаратов и редактирования информации о них.

Ключевыми методами этого модуля являются:

- searchMedicines() – реализует поиск лекарственных препаратов с учетом текстового запроса и выбранных фильтров;
- addMedicine() / editMedicine() / deleteMedicine() – реализует CRUD-операции по добавлению, редактированию и удалению препаратов из базы данных;
- onDoubleClick() – реализует функцию которая предоставляет возможность при двойном клике отправить выбранное лекарство в окно продаж.

Модуль использует название препарата, международное наименование, форму, производителя, категорию как входные данные для операций добавления и редактирования препаратов. При добавлении или редактировании взаимодействует с таблицей medicines, генерирует сигнал medicineSelected для передачи информации о выделенном препарате.

Взаимодействует с модулем базы данных. Также используется UserSession для предоставления доступа к окну только сотрудникам с ролью «Администратор».

2.2.4 Модуль складского учёта

Модуль (StockWindow) предназначен для управления партиями товара на складе: добавления, редактирования, удаления, поиска по названию или номеру партии.

Основные методы:

- loadStock() – производит загрузку записей из таблицы stock с соединением со справочниками;
- addStock() – реализует добавление новой партии лекарственного препарата;
- editStock() – метод для изменения параметров существующей партии;
- deleteStock() – осуществляет удаление партии;
- generateReport() – реализует выбор формата для генерации отчетов.

В качестве входных данных данных используются все поля партии. После добавления или редактирования данные заносятся в таблицу stock.

Модуль взаимодействует с модулем базы данных. Производится проверка прав через UserSession.

2.2.5 Модуль управления сотрудниками

Модуль (EmployeesWindow, AddEmployeeDialog) предназначен для просмотра, поиска, добавления, редактирования и удаления сотрудников.

Данный модуль состоит из таблицы сотрудников и диалога AddEmployeeDialog, предназначенного для добавления и редактирования информации о сотруднике.

Основные методы:

- loadEmployees() – осуществляет загрузку списка сотрудников;
- addEmployee() – осуществляет добавление нового сотрудника;
- editEmployee() – реализует обновление информации о сотруднике;
- deleteEmployee() – удаляет сотрудника.

Модуль взаимодействует с модулем базы данных. Доступ к модулю ограничен с помощью UserSession и доступен только сотрудникам, имеющим роль «Администратор».

2.2.6 Модуль истории продаж и отчётов

Модуль (SalesHistoryWindow) реализует просмотр истории продаж лекарственных препаратов. Просмотр продаж можно произвести за выбранный период, детализировать по конкретному чеку. Также можно сформировать отчет в форматах PDF, TXT, JSON.

Ключевыми методами для данного модуля являются:

- loadSales() – реализует загрузку списка продаж с фильтром по дате;
- onSaleSelected() – метод, реализующий просмотр состава продажи;
- generateReport() – реализует выбор формата для генерации отчетов.

Использует модуль базы данных для выборки данных. Доступ к модулю ограничен с помощью UserSession.

2.2.7 Модуль базы данных

Модуль (Database) представляет собой синглтон для управления подключением к SQLite и выполнения запросов.

Ключевыми методами модуля являются:

- connect() – реализует подключение к базе данных;
- disconnect() – реализует закрытие соединения с базой данных;
- createQuery() – создает объекта для выполнения SQL-запроса;
- loginUser() – метод предоставляющий проверку логина и пароля.

Данный модуль используется всеми модулями, которые используют данные из базы данных.

2.2.8 Модуль логирования

Модуль (Logger) представляет собой централизованный сбор событий системы (действия пользователя, ошибки, продажи, попытки входа). Поддерживает запись в разные файлы (actions, errors), ротацию по размеру, настраиваемый уровень детализации.

Основные методы:

- log() – реализует запись сообщения с уровнем логирования;
- logUserAction(), logLoginAttempt(), logSale(), logDatabaseQuery() – реализует специализированные логи;
- setLogLevel(), setMaxLogSize(), setMaxLogFiles() – реализует настройки;
- Макросы: LOG_DEBUG, LOG_INFO, LOG_WARNING, LOG_ERROR, LOG_CRITICAL.

Вызывается из других модулей для записи значимых событий.

2.2.9 Модуль настроек

Модуль (SettingsDialog), предоставляющий графический интерфейс для настройки параметров приложения: пути и формат логов, автоматическое создание отчётов, формат и оформление чеков.

Данный модуль содержит следующие вкладки:

- «Логи» – вкладка, в которой можно указать путь, уровень детализации, ротацию файлов логов, также можно включить использование отдельных файлов логов;
- «Отчёты» – вкладка, в которой можно указать путь, формат, автоматическое создание и детализацию отчетов;

- «Чеки» – вкладка, в которой можно путь, формат, автосохранение, автопечать, заголовок/подвал, количество копий чеков о продаже.

Ключевые методы:

- loadSettings() / saveSettings() – загрузка и сохранение через Qsettings.

Модуль сохраняет настройки в реестр/файл конфигурации. При применении может обновляет параметры Logger и других модулей (в текущей реализации – через applySettings).

2.2.10 Модуль генерации отчётов

Модуль (ReportGenerator) обеспечивает формирование структурированных отчётов по складу и по продажам в различных форматах. Реализован как отдельный сервисный класс, работающий в отдельном потоке, с возможностью отслеживания прогресса генерации.

2.2.10.1 Отчёт по складу

Модуль (Stock Report) формирует отчёт о наличии лекарственных средств на складе с возможностью фильтрации по категории и/или конкретному лекарству. Включает детальную информацию по каждой партии: название, категория, форма выпуска, производитель, номер партии, срок годности, количество, цены закупки и продажи, а также итоговые показатели.

Методы:

- generateStockReport() – точка входа для генерации отчёта по складу. Принимает путь к файлу, формат, идентификаторы и названия фильтров. В зависимости от формата вызывает соответствующий приватный метод;
- generateStockTxt() – формирует текстовый отчёт (.txt) с древовидной структурой, удобной для чтения человеком;
- generateStockJson() – генерирует структурированный JSON-файл, пригодный для машинной обработки и интеграции с другими системами;

- generateStockCsv() – создаёт CSV-файл для импорта в табличные процессоры;
- generateStockPdf() – генерирует HTML-представление с табличной вёрсткой и стилями.

2.2.10.2 Отчёт по продажам

Модуль (Sales Report) формирует сводку о продажах за заданный временной период. Включает список всех чеков с указанием номера, даты, суммы, сотрудника и способа оплаты, а также итоговые показатели.

Методы:

- generateSalesReport() – точка входа для генерации отчёта по продажам. Принимает путь к файлу, формат и диапазон дат;
- generateSalesTxt() – текстовый отчёт с перечислением чеков;
- generateSalesJson() – JSON-отчёт с массивом продаж и итоговыми агрегатами;
- generateSalesPdf() – HTML-отчёт с таблицей продаж.

2.2.11 Главный модуль

Модуль (Widget) осуществляет координацию работы всех окон, управление стеком виджетов, построением главного меню, инициализирует соединение с базой данных.

Ключевые методы модуля:

- setupMenu() – создает меню с пунктами «Файл», «Таблицы», «О программе»;
- setupStackedWidget() – настраивает переходы между окнами;
- onUserLoggedIn() – реализует обработку успешного входа; скрытие/показ пунктов меню в зависимости от роли пользователя;
- onCurrentWidgetChanged() – управление видимостью меню в зависимости от текущего окна.

Данный модуль создаёт экземпляры всех окон, подключает их сигналы и слоты, обеспечивает навигацию.

Графическое представление взаимосвязи модулей разрабатываемого продукта улучшит понимание целей и задач проекта. Так, представленная на рисунке 10 UML-диаграмма классов наглядно отображает логику взаимодействия модулей и раскрывает содержимое разрабатываемых классов.

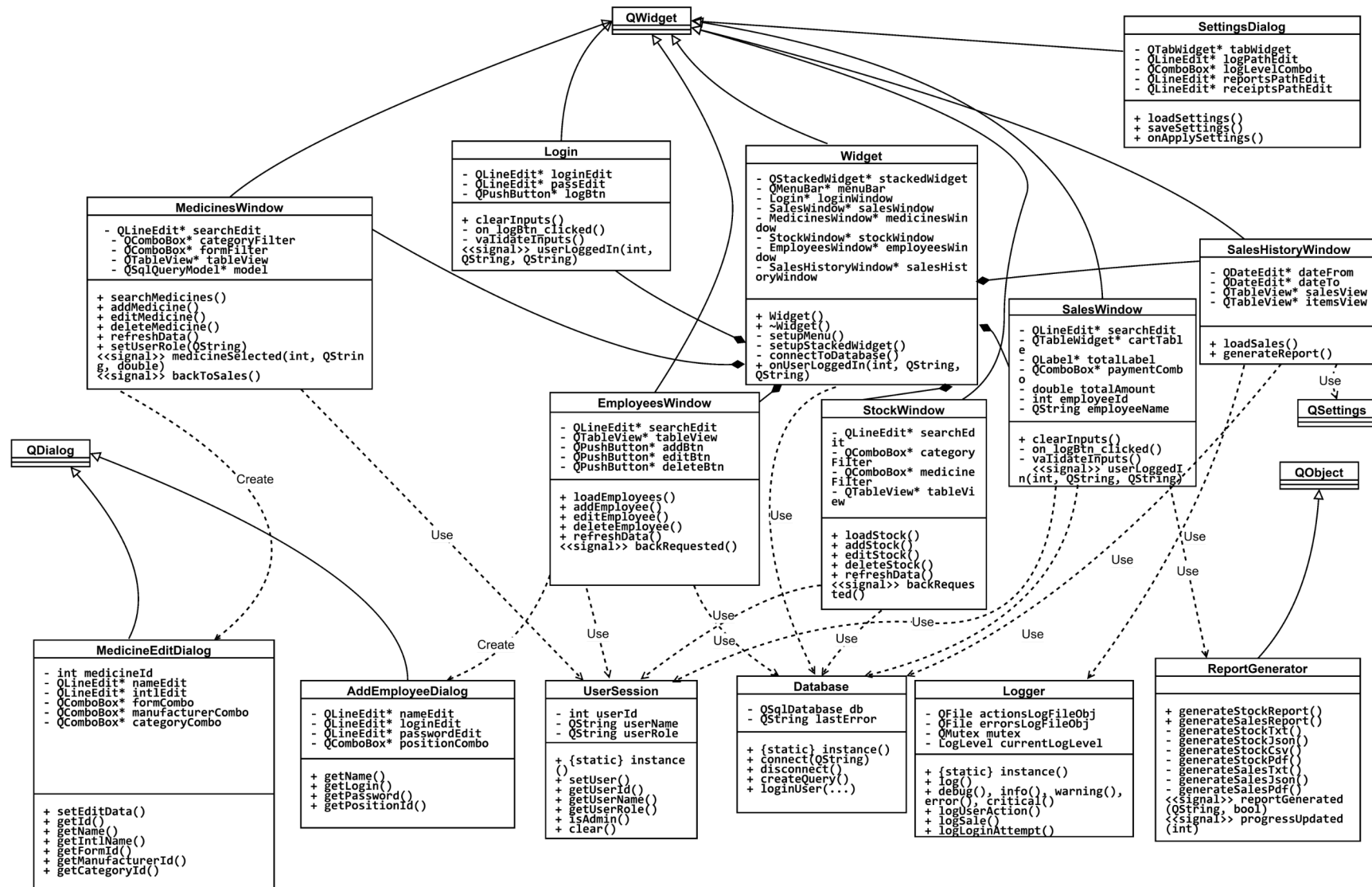


Рисунок 10 - UML-диаграмма классов

2.3 ИНТЕГРАЦИЯ ПРОГРАММНЫХ МОДУЛЕЙ

После описания назначения и внутреннего устройства каждого модуля необходимо рассмотреть, каким образом они объединяются в единую систему. Разработанная система построена по принципу слабой связанности модулей: каждый компонент отвечает за свою функциональную область и не имеет жёстких зависимостей от других. Однако для превращения набора независимых модулей в работающее приложение необходима их интеграция — организация взаимодействия, обмена данными и совместного использования ресурсов.

2.3.1 Механизмы взаимодействия модулей

В разработанной системе используются три основных механизма интеграции модулей, каждый из которых решает свою задачу.

2.3.1.1 Сигнально-слотовые соединения

Основным механизмом взаимодействия между окнами и виджетами являются сигнально-слотовые соединения. Они обеспечивают слабую связанность компонентов: отправитель сигнала не знает, кто его получает. Рассмотреть принцип работы этого механизма можно на примере соединения между окнами `MedicinesWindow` и `SalesWindow`.

При работе с аптечной системой пользователь (кассир) может открыть полный справочник лекарственных средств для удобного поиска необходимого препарата. Найдя нужное лекарство, пользователь двойным кликом мыши добавляет его в корзину текущей продажи. Для реализации этой функциональности используется сигнал `medicineSelected`, который передаёт три параметра: идентификатор складской записи, название препарата и цену.

Последовательность передачи данных выглядит следующим образом:

1. После двойного щелчка по строке таблицы `QTableView` генерируется встроенный сигнал `doubleClicked()`, который соединён со слотом `onDoubleClick()` класса `MedicinesWindow`.
2. В теле слота `onDoubleClick()` из модели извлекаются данные выбранной строки (идентификатор, название, цена).
3. Затем вызывается конструкция `emit medicineSelected(...)`, которая генерирует пользовательский сигнал с переданными параметрами.
4. Механизм сигналов и слотов `Qt` автоматически определяет все слоты, подключённые к данному сигналу, и последовательно вызывает их. В рассматриваемом случае это слот `addFromMedicines()` класса `SalesWindow`, который добавляет выбранное лекарство в корзину.

Ниже приведены фрагменты кода, реализующие описанное взаимодействие. Листинг 1 демонстрирует объявление сигнала в классе `MedicinesWindow`, листинг 2 — генерацию сигнала в слоте `onDoubleClick()`, листинг 3 — установку соединения между сигналом и слотом в главном модуле `Widget`.

Листинг 1 — Объявление сигнала в файле `medicineswindow.h`

1. `signals:`
2. `void medicineSelected(int stockId, QString name, double price);`

Листинг 2 — Генерация сигнала в методе `onDoubleClick()`, фрагмент файла `medicineswindow.cpp`

1. `void MedicinesWindow::onDoubleClick(const QModelIndex &index)`
2. `{`
3. `if (!index.isValid()) return;`
4.
5. `int stockId = model->data(model->index(row, 0)).toInt();`
6. `QString name = model->data(model->index(row, 1)).toString();`
7. `double price = model->data(model->index(row, 7)).toDouble();`
8.

```
9.         emit medicineSelected(stockId, name, price);
10.    }
```

Листинг 3 — Установка соединения в методе `setupStackedWidget()`, фрагмент файла `widget.cpp`

```
1.    connect(medicinesWindow, &MedicinesWindow::medicineSelected,
2.            salesWindow, &SalesWindow::addFromMedicines);
```

2.3.1.2 Синглтон-сервисы

Классы `Database`, `UserSession` и `Logger` реализованы как синглтоны, что обеспечивает единую точку доступа к ресурсам, согласованное состояние и удобство вызова из любого модуля без передачи ссылок.

Например, проверка прав пользователя может быть выполнена в любом окне системы вызовом фрагмента кода, показанного на листинге 4:

Листинг 4 — Проверка прав пользователя

```
1.    if (UserSession::instance().isAdmin()) {
2.        // Разрешить выполнение административной функции
3.    } else {
4.        // Запретить доступ
5.    }
```

2.3.1.3 Прямой вызов методов

Прямой вызов методов используется в случаях, когда модули явно зависят друг от друга и находятся в иерархии «родитель-потомок». Например, на листинге 5 показано, как главный класс `Widget` напрямую вызывает методы обновления данных у дочерних окон.

Листинг 5 — Прямой вызов метода refreshData окна EmployeesWindow, фрагмент файла widget.cpp

```
1. void Widget::openEmployeesWindow()
2. {
3.     employeesWindow->refreshData();        // Прямой вызов
4.     stackedWidget->setCurrentWidget(employeesWindow);
5. }
```

2.3.2 Совместное использование ресурсов

В процессе работы несколько модулей могут обращаться к одним и тем же ресурсам. Для предотвращения конфликтов и обеспечения целостности данных применяются следующие механизмы.

2.3.2.1 База данных (SQLite)

Доступ к базе данных осуществляется всеми модулями через единую точку входа - статический метод `Database::instance().createQuery()`, возвращающий объект запроса, связанный с текущим соединением. SQLite поддерживает одновременное выполнение нескольких запросов на чтение, однако запись блокируется на уровне файла базы данных, что исключает повреждение данных при конкурентных изменениях. Для обеспечения атомарности операций, затрагивающих несколько таблиц (например, одновременное создание записи о продаже, добавление позиций и списание остатков со склада), в методе `SalesWindow::checkout()` используются транзакции: при успешном выполнении всех операций транзакция фиксируется, при возникновении любой ошибки выполняется откат. После каждого выполнения SQL-запроса проверяется его успешность через метод `lastError()`, что позволяет своевременно выявлять и обрабатывать ошибки на уровне приложения.

Фрагмент кода, реализующий описанный механизм транзакционной обработки продажи, приведён в листинге 6.

Листинг 6 — Транзакционная обработка оформления продажи

```
1. void SalesWindow::checkout()
2. {
3.     // ... проверка корзины и подготовка данных ...
4.
5.     // Начинаем транзакцию
6.     QSqlDatabase db = QSqlDatabase::database();
7.     db.transaction();
8.
9.     // 1. Создаём запись о продаже
10.    QSqlQuery query = Database::instance().createQuery();
11.    query.prepare("INSERT INTO sales (sale_number, employee_id,
        final_amount, payment_method_id) "
12.        "VALUES (:num, :emp, :amount, :pay)");
13.    // ... привязка параметров ...
14.
15.    if (!query.exec()) {
16.        db.rollback(); // Откат при ошибке
17.        LOG_ERROR("Не удалось создать продажу: " +
            query.lastError().text());
18.        return;
19.    }
20.
21.    int saleId = query.lastInsertId().toInt();
22.
23.    // 2. Добавляем позиции продажи и обновляем склад
24.    for (int i = 0; i < cartTable->rowCount(); i++) {
25.        // ... добавление позиции в sale_items ...
26.        // ... обновление остатка в stock ...
27.        if (!itemQuery.exec() || !updateQuery.exec()) {
```

```

28.         db.rollback(); // Откат при любой ошибке
29.         LOG_ERROR("Ошибка при добавлении позиции: " +
    itemQuery.lastError().text());
30.         return;
31.     }
32. }
33.
34. // Транзакция фиксируется
35. if (!db.commit()) {
36.     db.rollback();
37.     LOG_ERROR("Не удалось завершить транзакцию");
38.     return;
39. }
40.
41. // ... генерация чека и очистка корзины ...
42. }

```

2.3.2.2 Файлы логов.

Логирование осуществляется исключительно через класс `Logger`, реализованный как синглтон. Это гарантирует, что все модули записывают события в едином формате и через единый интерфейс. Для обеспечения потокобезопасности при записи логов из разных потоков (например, при асинхронной генерации отчёта) используется объект `QMutex`, который блокирует запись на время выполнения операции, тем самым предотвращая перемешивание сообщений и повреждение файлов. Реализован механизм ротации логов: при превышении заданного максимального размера файла (по умолчанию 10 МБ) текущий файл переименовывается с добавлением временной метки, а на его месте создаётся новый. Количество хранимых архивных файлов также ограничено (по умолчанию 5). Предусмотрена возможность раздельного хранения логов действий пользователей и логов

ошибок: они могут записываться в файлы actions_*.log и errors_*.log соответственно, что упрощает анализ инцидентов.

Фрагмент кода, демонстрирующий потокобезопасную запись логов и механизм ротации, приведён в листинге 7.

Листинг 7 — Потокобезопасная запись логов с ротацией файлов, фрагмент файла logger.cpp

```
1. void Logger::writeToActionsLog(const QString &formattedMessage)
2. {
3.     // Блокировка мьютекса для потокобезопасной записи
4.     QMutexLocker locker(&mutex);
5.
6.     if (!isInitialized || !actionsLogFileObj.isOpen()) {
7.         std::cout << "[ACTIONS] " << formattedMessage.toStdString()
8.         << std::endl;
9.         return;
10.    }
11.    actionsLogStream << formattedMessage << "\n";
12.    actionsLogStream.flush();
13.
14.    // Проверка размера файла и ротация при необходимости
15.    checkFileSize(actionsLogFileObj, actionsLogFile);
16. }
17.
18. void Logger::checkFileSize(QFile &file, const QString &filePath)
19. {
20.     if (file.size() > maxLogSize) {
21.         // Временно освобождаем мьютекс перед ротацией
22.         mutex.unlock();
23.
24.         rotateLogFile(file, (filePath == actionsLogFile) ?
                actionsLogStream : errorsLogStream, filePath);
```

```

25.
26.     mutex.lock();
27.     }
28. }
29.
30. void Logger::rotateLogFile(QFile &file, QTextStream &stream, const
    QString &filePath)
31. {
32.     if (!file.isOpen()) return;
33.
34.     stream.flush();
35.     file.close();
36.
37.     // Переименовываем текущий файл с добавлением временной метки
38.     QString backupName = filePath + "." +
39.     QDateTime::currentDateTime().toString("yyyyMMdd-hhmmss") +
        ".bak";
40.     QFile::rename(filePath, backupName);
41.
42.     // Открываем новый файл
43.     file.setFileName(filePath);
44.     if (file.open(QIODevice::WriteOnly | QIODevice::Append |
        QIODevice::Text)) {
45.         stream.setDevice(&file);
46.         stream << "\n=== Лог-файл создан: "
47.         << QDateTime::currentDateTime().toString("yyyy-MM-dd hh:mm:ss")
        << " ===\n";
48.         stream.flush();
49.     }
50. }

```

2.3.2.3 Пользовательская сессия.

Класс `UserSession` реализован как синглтон и хранит информацию о текущем авторизованном пользователе: идентификатор, имя и роль. Доступ к сессии осуществляется через `UserSession::instance()` из любого модуля без необходимости передавать ссылку на объект между окнами. Поскольку все операции, связанные с сессией (установка, получение, проверка прав), выполняются в главном GUI-поток, состояние гонки невозможно — данные всегда согласованы. При выходе пользователя из системы вызывается метод `clear()`, который сбрасывает все хранимые значения, что предотвращает несанкционированный доступ к функциям, требующим авторизации, после завершения сеанса.

Пример использования класса `UserSession` для проверки прав доступа приведён в листинге 8.

Листинг 8 — Проверка прав доступа перед выполнением административной операции, фрагмент файла `employeeswindow.cpp`

```
1. void EmployeesWindow::deleteEmployee()
2. {
3.     // Проверка прав: только администратор может удалять
    сотрудников
4.     if (!UserSession::instance().isAdmin()) {
5.         QMessageBox::warning(this, "Ошибка", "Недостаточно прав для
    выполнения операции");
6.         return;
7.     }
8.
9.     QModelIndex index = tableView->currentIndex();
10.    if (!index.isValid()) return;
11.
12.    int id = model->data(model->index(index.row(), 0)).toInt();
13.
```

```

14.     QSqlQuery query = Database::instance().createQuery();
15.     query.prepare("DELETE FROM employees WHERE id = :id");
16.     query.bindValue(":id", id);
17.
18.     if (query.exec()) {
19.         loadEmployees();
20.         LOG_INFO("Сотрудник удалён пользователем " +
                UserSession::instance().getUserName());
21.     }
22. }

```

2.3.2.4 Файлы чеков и отчётов.

Для исключения конфликтов при одновременном создании файлов (например, при оформлении нескольких продаж подряд) применяется механизм генерации уникальных имён. В качестве основы используется временная метка с точностью до миллисекунд, что гарантирует уникальность имени даже при интенсивной работе. Пример сформированного имени: receipt_20241225_143022.txt. Перед записью файла программа проверяет существование целевой директории с помощью метода QDir().mkpath() и, при необходимости, создаёт её автоматически. Это избавляет пользователя от ручного создания каталогов для хранения чеков и отчётов. Благодаря использованию временной метки с высокой точностью коллизии имён полностью исключены, что позволяет безопасно выполнять параллельные операции записи из разных потоков или окон приложения.

Фрагмент кода, реализующий генерацию уникального имени файла и автоматическое создание директории, приведён в листинге 9.

Листинг 9 — Генерация уникального имени файла чека, фрагмент файла saleswindow.cpp

```

1. void SalesWindow::checkout()

```

```

2.  {
3.      // ... оформление продажи ...
4.
5.      // Генерация уникального имени файла на основе временной метки
6.      QString saleNumber = "SALE-" +
    QDateTime::currentDateTime().toString("yyyyMMddHHmmss");
7.      QString receiptsPath = QDir::currentPath() + "/receipts/";
8.
9.      // Автоматическое создание директории, если она не существует
10.     QDir().mkpath(receiptsPath);
11.
12.     // Формирование полного имени файла
13.     QString fileName = receiptsPath + "receipt_" + saleNumber +
    ".txt";
14.
15.     // Сохранение чека
16.     QFile file(fileName);
17.     if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
18.         QTextStream out(&file);
19.         out << receiptText;
20.         file.close();
21.         LOG_INFO(QString("Чек сохранён: %1").arg(fileName));
22.     } else {
23.         LOG_ERROR(QString("Ошибка сохранения чека:
    %1").arg(file.errorString()));
24.     }
25. }

```

2.3.3 Обработка ошибок при интеграции

В процессе интеграции программных модулей необходимо предусмотреть корректную реакцию системы на различные штатные и нештатные ситуации, возникающие на стыках компонентов. В разработанной аптечной информационной системе реализованы механизмы обработки следующих типов ошибок.

2.2.3.1 Отсутствие подключения к базе данных.

Данная ошибка обнаруживается на этапе инициализации приложения в методе `Database::connect()` при попытке открытия соединения с SQLite-файлом. Если подключение не может быть установлено, метод возвращает `false`, а в классе `Widget` генерируется исключение. При перехвате исключения пользователю отображается критическое сообщение через `QMessageBox::critical()`, после чего приложение завершает свою работу. Такой подход предотвращает запуск системы в неработоспособном состоянии.

Фрагмент кода, реализующий описанный механизм обработки ошибки подключения к базе данных, приведён в листинге 10.

Листинг 10 — Обработка ошибки подключения к базе данных

```
1. //database.cpp
2. bool Database::connect(const QString &path)
3. {
4.     if (QSqlDatabase::contains("qt_sql_default_connection"))
5.         db = QSqlDatabase::database("qt_sql_default_connection");
6.     else
7.         db = QSqlDatabase::addDatabase("SQLITE");
8.
9.     db.setDatabaseName(path);
10.
11.    if (!db.open()) {
```



```

12.         lastError = db.lastError().text();
13.         return false; // Возвращаем false при ошибке подключения
14.     }
15.
16.     qDebug() << "БД подключена";
17.     return true;
18. }
19.
20. // widget.cpp
21. void Widget::connectToDatabase()
22. {
23.     QString dbPath = "../resources/Database.db";
24.
25.     if (!Database::instance().connect(dbPath)) {
26.         throw std::runtime_error("Ошибка подключения к БД: " +
27.             Database::instance().getLastError().toString());
28.     }
29. }
30. // widget.cpp (конструктор Widget)
31. Widget::Widget(QWidget *parent) : QWidget(parent)
32. {
33.     try {
34.         connectToDatabase(); // Может выбросить исключение
35.
36.         // ... остальная инициализация ...
37.     }
38.     catch (const std::exception &e) {
39.         QMessageBox::critical(this, "Ошибка", e.what());
40.         QApplication::quit(); // Завершение приложения
41.     }
42. }

```

2.2.3.2 Неверный логин или пароль.

Ошибка аутентификации возникает в методе `Database::loginUser()` при несовпадении введенных учетных данных с записями в базе данных. Метод возвращает `false`, а текст ошибки сохраняется в переменной `lastError`. Модуль `Login`, получив отрицательный результат, отображает пользователю сообщение через `QMessageBox::warning()` с текстом, полученным из `Database::instance().getLastError()`. Пользователю предлагается повторить ввод, при этом поля ввода не очищаются полностью, а только выделяются для корректировки.

2.2.3.3 Ошибка выполнения SQL-запроса.

Любой модуль, выполняющий запрос к базе данных, проверяет результат вызова `query.exec()`. При неудачном выполнении (например, синтаксическая ошибка в SQL или нарушение ограничений целостности) происходит следующее: во-первых, в лог-файл записывается сообщение об ошибке через макрос `LOG_ERROR()` с указанием текста ошибки из `query.lastError().text()`; во-вторых, пользователю отображается диалоговое окно с сообщением о невозможности выполнения операции; в-третьих, текущая операция прерывается без изменения данных.

2.2.3.4 Нарушение внешнего ключа при удалении записи.

При попытке удаления лекарственного средства, которое уже использовалось в продажах, возникает ошибка внешнего ключа, так как на запись в таблице `medicines` ссылаются записи из таблицы `sale_items`. В методе `MedicinesWindow::deleteMedicine()` проверяется текст ошибки: если он содержит подстроку `"FOREIGN KEY"`, пользователю выводится понятное предупреждение: «Нельзя удалить лекарство — оно используется в продажах». Запись при этом не удаляется. Аналогичная логика реализована для удаления партий со склада. Пример реализации показан в листинге 11:

Листинг 11 — Обработка ошибки внешнего ключа при удалении записи

```
1.  if (!query.exec()) {
2.      QString err = query.lastError().text();
3.      if (err.contains("FOREIGN KEY")) {
4.          QMessageBox::warning(this, "Ошибка",
            "Нельзя удалить запись — она используется в других таблицах");
5.      } else {
6.          QMessageBox::critical(this, "Ошибка", err);
7.      }
8.      return;
9.  }
```

2.2.3.5 Недостаточно прав для выполнения операции.

Перед выполнением административных функций (открытие окна сотрудников, истории продаж, склада, а также операции добавления, редактирования и удаления записей) выполняется проверка роли текущего пользователя через `UserSession::instance().isAdmin()`. Если пользователь не является администратором, операция отменяется, вызывается `QMessageBox::warning()` с сообщением «Недостаточно прав», а соответствующие кнопки интерфейса скрываются (или становятся недоступными) сразу после входа в систему. Это обеспечивает защиту критических функций на двух уровнях: на уровне пользовательского интерфейса и на уровне бизнес-логики.

2.2.3.6 Нехватка товара на складе при оформлении продажи.

В методе `SalesWindow::checkout()` перед списанием остатков выполняется проверка доступного количества каждой позиции в корзине. Если запрашиваемое количество превышает остаток на складе, инициируется откат открытой ранее транзакции с помощью `db.rollback()`. Пользователю

отображается сообщение, содержащее информацию о том, какого товара недостаточно и какое количество доступно. Продажа не оформляется, корзина остаётся неизменной, что позволяет пользователю скорректировать количество или удалить позицию.

2.2.3.7 Не удалось сохранить чек.

После успешного оформления продажи система пытается сохранить файл чека в директории `receipts/`. Если операция записи файла не удалась (например, недостаточно прав на запись в директорию или закончилось место на диске), ошибка фиксируется в логе через `LOG_ERROR()` с указанием причины (`file.errorString()`). При этом сама продажа считается успешно завершённой, так как основные данные уже сохранены в базе данных. Пользователю не выводится сообщение об ошибке, чтобы не создавать впечатления, что продажа не состоялась, однако факт проблемы фиксируется для последующего анализа системным администратором.

2.2.3.8 Отсутствие директории для логов.

При инициализации системы логирования в конструкторе класса `Logger` проверяется существование директории, указанной в настройках для хранения лог-файлов. Если директория отсутствует, она автоматически создаётся с помощью метода `QDir().mkpath()`. Аналогичная логика реализована перед созданием файлов чеков и отчётов в модулях `SalesWindow` и `SalesHistoryWindow`. Такой подход полностью исключает ошибки, связанные с отсутствием целевых каталогов, и избавляет пользователя от необходимости ручного создания директорий.

2.2.3.9 Обобщённый механизм обработки ошибок.

Во всех описанных случаях соблюдаются следующие принципы: критические ошибки (отсутствие БД) приводят к остановке приложения; ошибки, связанные с некорректными действиями пользователя (неверный

пароль, недостаточно прав, нехватка товара), обрабатываются с информированием пользователя через диалоговые окна; технические ошибки (проблемы с записью файлов) логируются, но не прерывают основной рабочий процесс. Такой подход обеспечивает баланс между надёжностью системы и удобством её использования.

Пример диалогового окна, информирующего пользователя о некорректном действии, приведён в листинге 12.

Листинг 12 — Вывод диалогового окна с предупреждением при недостатке прав

```
1. void EmployeesWindow::deleteEmployee()
2. {
3.     // Проверка прав доступа
4.     if (!UserSession::instance().isAdmin()) {
5.         QMessageBox::warning(this,
6.             "Ошибка доступа", "Недостаточно прав для удаления
сотрудника");
7.         return;
8.     }
9.
10.    // ... дальнейшее выполнение операции ...
11. }
```

В данном разделе приведены лишь ключевые фрагменты кода, демонстрирующие механизмы взаимодействия и обработки ошибок. Полные листинги всех модулей системы вынесены в Приложение А.

2.4 ОПИСАНИЕ ПРОГРАММНОГО ИНТЕРФЕЙСА

Главное окно приложения представляет собой контейнер, который переключается между различными экранами в зависимости от текущего режима работы. Верхняя часть окна содержит главное меню, нижняя — строку состояния. Центральная область отводится под активное окно.

Ниже приводится описание окон системы: их назначение, внешний вид и основные элементы управления. Для каждого окна указано, для каких операций оно предназначено, какие данные отображает и какие действия может выполнять пользователь.

2.4.1 Окно авторизации

Назначение: Точка входа в систему. Используется для идентификации и аутентификации пользователей. Здесь вам необходимо ввести свои учётные данные, чтобы система определила вашу роль и предоставила доступ к соответствующим функциям.

Как открыть: Данное окно открывается автоматически при запуске приложения. Оно закроется только после успешного входа в систему или завершения работы программы.

Внешний вид и управление: Окно содержит заголовок, поля для ввода логина и пароля, а также кнопку «Войти».

Чтобы войти в систему введите свой логин и пароль в соответствующие поля и нажмите кнопку «Войти». Система проверит введённые учётные данные и предоставит доступ.

Внешний вид окна показан на рисунке 11.

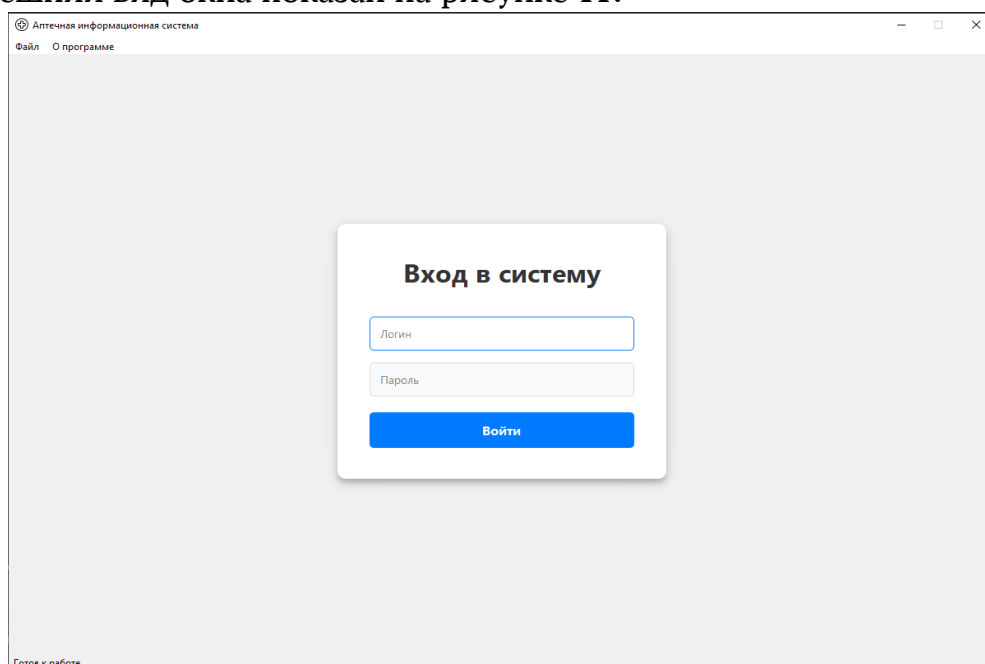


Рисунок 11 — Внешний вид окна авторизации

2.4.2 Окно продаж

Назначение: Основное рабочее окно для кассира аптеки. Предназначено для оформления продажи лекарственных препаратов.

Действия: С помощью этого окна вы можете искать товары, добавлять их в корзину, выбирать способ оплаты и завершать сделку, формируя чек.

Окно содержит следующие элементы:

- Поле поиска: служит для быстрой фильтрации товаров.
- Таблица доступных лекарств: отображает название, цену и остаток товаров.
- Кнопка добавления в корзину: помещает выбранный товар в корзину покупателя.
- Таблица корзины: отображает список добавленных товаров. Здесь можно изменять их количество.
- Кнопка удаления из корзины: удаляет выбранную позицию из корзины.
- Выпадающий список: позволяет выбрать способ оплаты.
- Метка итоговой суммы: отображает общую стоимость покупки.
- Кнопка «Оплатить»: завершает оформление продажи. При нажатии система проверяет наличие товара на складе, списывает остатки, сохраняет запись о продаже в базе данных и формирует чек.

Внешний вид окна показан на рисунке 12.

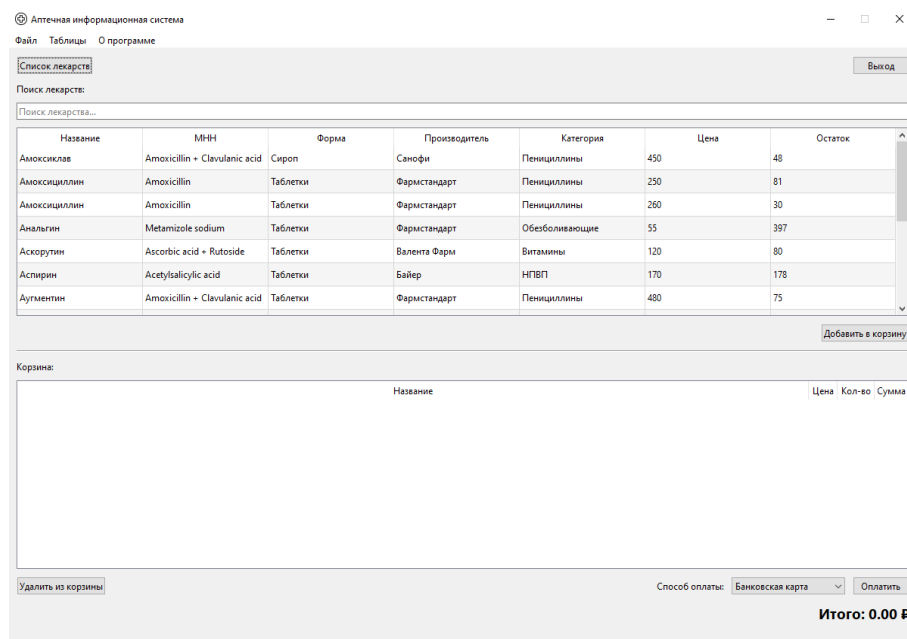


Рисунок 12 — Внешний вид окна продаж

2.4.3 Окно справочника лекарств

Назначение: Просмотр и редактирование списка лекарственных средств. Позволяет администратору управлять ассортиментом.

Действия: Вы можете добавлять новые препараты, изменять информацию о существующих или удалять устаревшие записи.

Как открыть: Окно открывается из главного меню или по кнопке «Список лекарств» из окна продаж.

Окно содержит следующие элементы:

- Фильтры: поле поиска и выпадающие списки.
- Таблица лекарств: отображает название, МНН, форму, производителя, категорию, цены и остаток.

Кнопки действий:

- «Добавить»: открывает диалоговое окно для ввода информации о новом лекарстве.
- «Редактировать»: открывает диалоговое окно для изменения данных выбранного лекарства.
- «Удалить»: удаляет выбранную запись.
- «Назад»: возвращает к предыдущему окну.

Внешний вид окна показан на рисунке 13.

Аптечная информационная система

Файл Таблицы О программе

Назад

Поиск (название / международное)

Категория: Все категории Форма: Все формы

	Название	МНН	Форма	Производитель	Категория	Закупка	Продажа	Остаток
1	Амоксилав	Amoxicillin + Clavulani...	Сироп	Санофи	Пенициллины	280	450	48
2	Амоксициллин	Amoxicillin	Таблетки	Фармстандарт	Пенициллины	150	250	81
3	Амоксициллин	Amoxicillin	Таблетки	Фармстандарт	Пенициллины	155	260	30
4	Анальгин	Metamizole sodium	Таблетки	Фармстандарт	Обезболивающие	25	55	397
5	Аскорутин	Ascorbic acid + Rutoside	Таблетки	Валента Фарм	Витамины	65	120	80
6	Аспирин	Acetylsalicylic acid	Таблетки	Байер	НПВП	90	170	178
7	Аугментин	Amoxicillin + Clavulani...	Таблетки	Фармстандарт	Пенициллины	320	480	75
8	Валидол	Validol	Таблетки	Валента Фарм	Сердечно-сосудистые	35	70	90
9	Ибупрофен	Ibuprofen	Таблетки	Новартис	НПВП	145	240	160
10	Лоратадин	Loratadine	Таблетки	Эгис	Антигистаминные	45	95	249
11	Нурофен	Ibuprofen	Таблетки	Байер	НПВП	180	290	200
12	Нурофен	Ibuprofen	Таблетки	Байер	НПВП	175	285	150
13	Парацетамол	Paracetamol	Таблетки	Санофи	Жаропонижающие	80	150	299
14	Супрастин	Chloropyramine	Таблетки	Пфайзер	Антигистаминные	220	350	126

Добавить Редактировать Удалить

Рисунок 13 — Внешний вид окна справочника лекарств

2.4.4 Окно управления складом

Назначение: Учёт партий лекарственных средств. Позволяет администратору контролировать складские запасы.

Действия: Вы можете добавлять новые партии, корректировать количество, цены и сроки годности, а также удалять устаревшие или испорченные товары.

Доступ: Окно доступно только пользователям с ролью «Администратор».

Как открыть: Открывается из главного меню.

Окно содержит следующие элементы:

- **Фильтры:** поле поиска (по названию лекарства или номеру партии) и выпадающие списки (по категории и конкретному лекарству).
- **Таблица партий:** отображает название лекарства, номер партии, срок годности, количество, цены и дату поступления.

Кнопки действий:

- «Добавить партию»: открывает диалоговое окно для ввода информации о новой партии.
- «Редактировать»: открывает диалоговое окно для изменения данных выбранной партии.
- «Удалить»: удаляет выбранную партию.
- «Обновить»: обновляет данные в таблице.
- «Назад»: возвращает к предыдущему окну.

Внешний вид окна показан на рисунке 14.

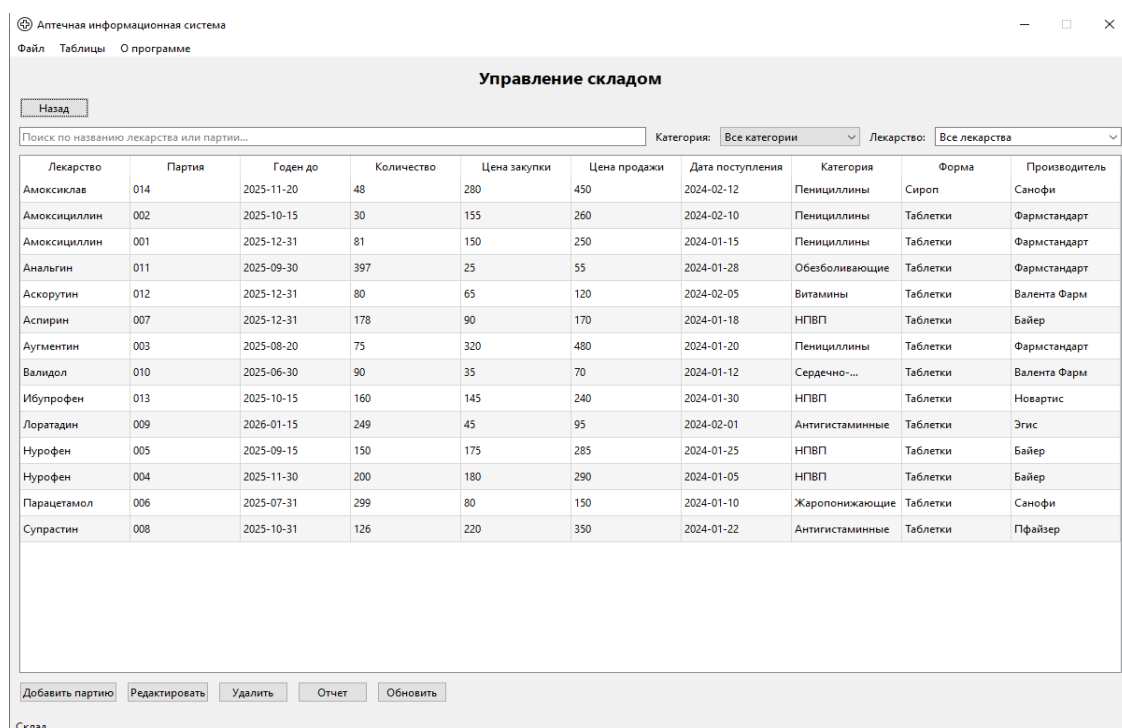


Рисунок 14 — Внешний вид окна управления складом

2.4.5 Окно управления сотрудниками

Назначение: Администрирование учётных записей работников аптеки.

Действия: Вы можете добавлять новых сотрудников, изменять их данные (ФИО, логин, пароль, должность) или удалять учётные записи уволенных сотрудников.

Доступ: Окно доступно только пользователям с ролью «Администратор».

Как открыть: Открывается из главного меню.

Окно содержит следующие элементы:

- Поле поиска: фильтрация по фамилии сотрудника.
- Таблица сотрудников: отображает ФИО, должность и логин.
- Кнопки действий: «Добавить», «Редактировать», «Удалить» работают по тому же принципу, открывая диалоговые окна для ввода или изменения данных.
- Кнопки «Обновить» и «Назад»: для обновления списка и возврата в предыдущее меню.

Внешний вид окна показан на рисунке 15.

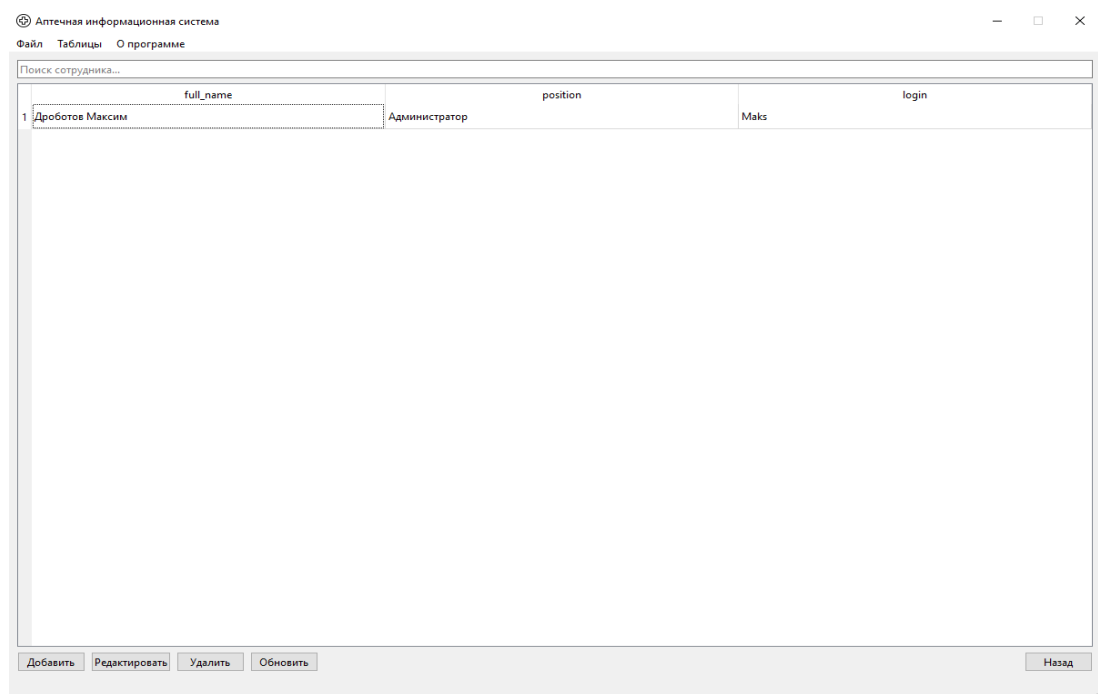


Рисунок 15 — Внешний вид окна управления сотрудниками

2.4.6 Окно истории продаж

Назначение: Просмотр архивных продаж и формирование отчётов.

Действия: Вы можете выбрать период, просмотреть список продаж и детальный состав каждого чека, а также сохранить отчёт в удобном формате.

Доступ: Окно доступно только пользователям с ролью «Администратор».

Как открыть: Открывается из главного меню.

Окно содержит следующие элементы:

- Панель выбора периода: два поля для ввода дат «от» и «до».
- Таблица продаж: отображает номер продажи, дату, сумму, сотрудника и способ оплаты.
- Таблица состава чека: показывает детализацию выбранной в верхней таблице продажи (название, количество, цена, сумма).
- Кнопки действий:
- «Обновить»: применяет выбранный фильтр по дате.
- «Сформировать отчёт»: позволяет сохранить отчёт за выбранный период.

После нажатия программа предложит выбрать формат файла (TXT, PDF, JSON) и сохранит его в папку отчётов.

Внешний вид окна показан на рисунке 16.

Аптечная информационная система
Файл Таблицы О программе

История продаж

С: 27.03.2026 По: 26.04.2026 Обновить Сформировать отчет

Продажи:

Номер продажи	Дата	Сумма	Сотрудник	Способ оплаты
SALE-2026042421211	2026-04-24 19:12:11	1240	Неизвестный	Банковская карта
SALE-2026042420520	2026-04-24 19:05:20	1435	Дроботов Максим	Банковская карта
1774976242	2026-03-31 16:57:22	245	Дроботов Максим	Наличные
1774892119	2026-03-30 17:35:19	3570	Дроботов Максим	Наличные
1774881208	2026-03-30 14:33:28	3320	Дроботов Максим	Банковская карта
1774802044	2026-03-29 16:34:04	1790	Дроботов Максим	Наличные

Состав продажи:

Название	Количество	Цена	Сумма
Амоксициллин	6	250	1500
Амоксициллин	7	260	1820

Рисунок 16 — Внешний вид окна истории продаж

2.4.7 Окно настроек

Назначение: Конфигурирование параметров работы приложения.

Действия: Вы можете изменить пути сохранения логов, отчётов и чеков, уровень детализации логирования, форматы выходных файлов, а также настройки оформления чеков.

Доступ: Окно доступно всем авторизованным пользователям.

Как открыть: Главное меню → Файл → Настройки.

Внешний вид и управление:

Окно содержит три вкладки: «Логи», «Отчёты» и «Чеки». Каждая вкладка предлагает свой набор параметров: текстовые поля для указания путей сохранения, выпадающие списки для выбора форматов, числовые счётчики и флажки для включения опций.

Кнопки действий:

- «Применить» или «ОК»: сохраняет введённые настройки и применяет их ко всем модулям системы.
- «Сбросить»: возвращает настройки к исходным значениям.
- «Отмена»: закрывает окно без сохранения изменений.

Внешний вид окна показан на рисунке 17.

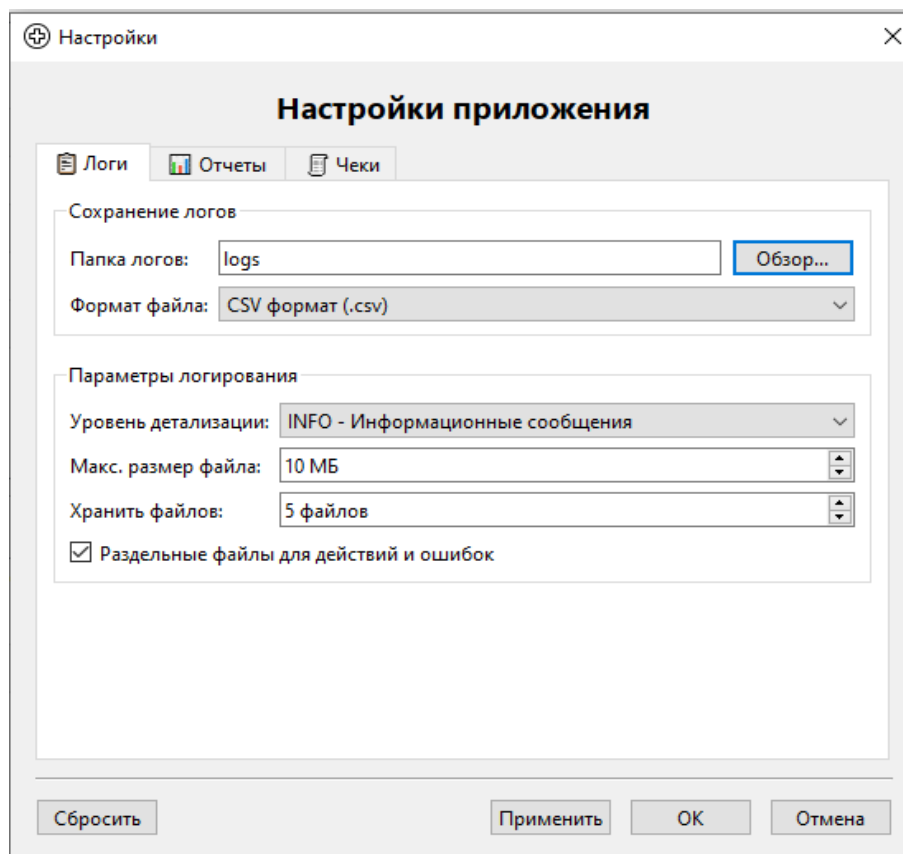


Рисунок 17 — Внешний вид окна настроек

2.4.8 Обратная связь и сообщения пользователю

Система использует несколько типов уведомлений, чтобы информировать вас о результате ваших действий, возникающих ошибках или запрашивать подтверждение критических операций:

- Информационные сообщения: отображаются в строке состояния в нижней части окна и автоматически исчезают через несколько секунд. Примеры: приветствие пользователя, уведомление об успешной продаже.
- Предупреждения: выводятся во всплывающем окне при некорректных действиях. Примеры: попытка выполнить операцию без прав доступа, пустая корзина, нехватка товара на складе.
- Критические ошибки: выводятся во всплывающем окне при серьёзных сбоях в работе программы (например, отсутствие подключения к базе данных).

- Запросы подтверждения: выводятся во всплывающем окне с вариантами ответа («Да»/«Нет») перед необратимыми операциями. Примеры: подтверждение оплаты, удаление записи из справочника.

Пример диалогового окна с предупреждением, которое появляется при попытке входа в систему с пустым полем логина, показан на рисунке 18.

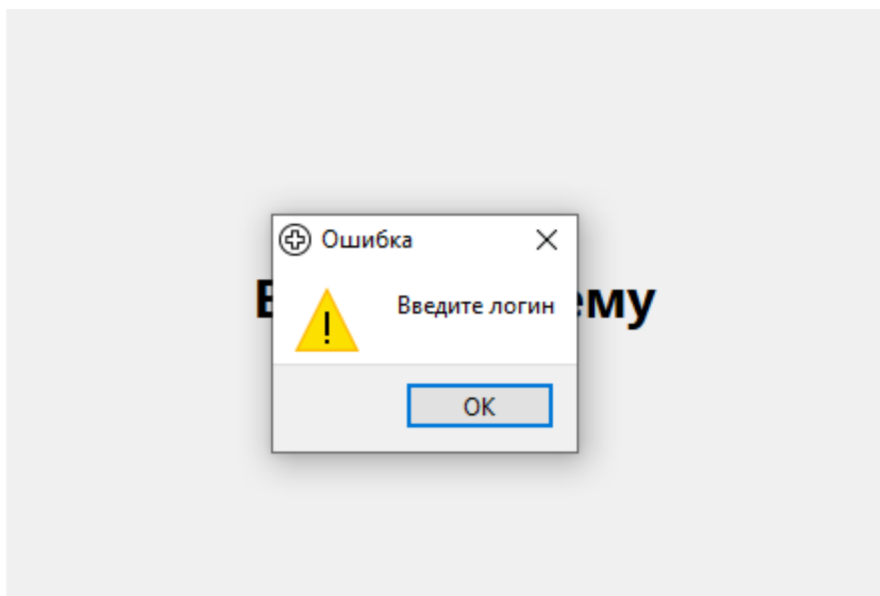


Рисунок 18 — Пример диалогового окна-предупреждения

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была достигнута поставленная цель — разработано программное обеспечение для автоматизации учёта продаж лекарственных средств в аптеке, представляющее собой кроссплатформенное настольное приложение на базе фреймворка Qt и СУБД SQLite.

Для достижения цели были последовательно решены все сформулированные задачи. Проведён анализ предметной области «Аптека», выделены ключевые бизнес-процессы и информационные потоки, определены основные сущности и их взаимосвязи. Построенная модель бизнес-процессов в нотации BPMN позволила наглядно представить работу аптеки до и после автоматизации, обосновав тем самым необходимость внедрения программного продукта.

На основе анализа были сформулированы функциональные и нефункциональные требования к системе, охватывающие аспекты безопасности, производительности, удобства использования и отказоустойчивости. Спроектирована и реализована реляционная база данных, ER-диаграмма которой включает все необходимые сущности: лекарственные средства, складские партии, продажи, сотрудников и справочники. Схема базы данных обеспечивает целостность данных на уровне внешних ключей и соответствует третьей нормальной форме.

Разработанное приложение состоит из девяти функционально независимых модулей, объединённых в единую систему. Модуль авторизации обеспечивает безопасный вход с хешированием паролей по алгоритму SHA-256. Модуль продаж реализует основную бизнес-функцию — оформление продажи с транзакционной обработкой данных, списанием остатков и автоматической генерацией чеков. Модули справочника лекарств и складского учёта предоставляют CRUD-операции с возможностью фильтрации и поиска. Модуль истории продаж позволяет анализировать архивные данные и формировать

отчёты в форматах TXT, JSON, CSV и HTML. Вспомогательные модули логирования и настроек обеспечивают мониторинг работы системы и гибкую конфигурацию без изменения исходного кода.

Реализованная архитектура соответствует паттерну Model-View, что обеспечивает разделение бизнес-логики и пользовательского интерфейса. Механизм сигналов и слотов Qt гарантирует слабую связанность компонентов и упрощает расширение функциональности. Все операции, изменяющие состояние базы данных, выполняются в транзакциях с возможностью полного отката при возникновении ошибок, что исключает нарушение целостности данных.

Тестирование программного продукта подтвердило его работоспособность и соответствие заявленным требованиям. Модульные тесты охватили критически важные алгоритмы расчёта стоимости и валидации данных, функциональное тестирование проверило корректность работы пользовательского интерфейса и взаимодействия между модулями.

Практическая значимость работы заключается в создании готового к внедрению инструмента, позволяющего аптечным организациям сократить временные затраты на рутинные операции, минимизировать ошибки учёта, связанные с человеческим фактором, и обеспечить оперативный контроль сроков годности и остатков товара. Расчёт показателей экономической эффективности показал, что срок окупаемости программного продукта составляет около 1,2 года, а годовой экономический эффект достигает 304 645 рублей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Рахматуллин Р. Р. Расчет технико-экономических показателей и определение экономического эффекта программного продукта: методические указания к расчетно-графическому заданию, курсовому и дипломному проектированию / Р.Р. Рахматуллин, Л.Ф. Давлетбаева. – Оренбург: ООО «Агентство «Пресса», 2008.
2. Федорова Г. Н. Сопровождение информационных систем: учебник для студ. Учреждений сред. Проф. Образования / Г. Н. Федорова. — М.: Изд. центр «Академия», 2018
3. Юнит-тестирование: полное руководство. — Текст : электронный // QaRocks : [сайт]. — URL: <https://qarocks.ru/unit-testing-tutorial/> (дата обращения: 05.02.2026).

ПРИЛОЖЕНИЕ А

Листинги программного кода

<В Содержании должно быть так:

ПРИЛОЖЕНИЕ А Листинги программного кода.....N

Чтобы это сделать нужно включить правку Содержания: Из контекста → Edit
Оглавление → Снять флажок: «Защищено от изменения в ручную» и выполнить
правку Содержания вручную. Поскольку Содержание будет постоянно
меняться, необходимо редактирование Содержания произвести в самую
последнюю очередь>

Листинг 13 — Образец оформления Листинга программного кода

12. <Внутри таблицы без видимой границы находится программный код с
нумерацией>

<Как сделать:>

<Таблица → Вставить таблицу; столбцов 1, строк 1>

<Убираем видимость границ. Контекстное меню → Свойство таблицы →
Обрамление, снимаем. Другие свойства не изменяем!>

<Вставка → Название → Категория: Листинг. Счетчик меняет значения
автоматически>

<Например>

Листинг 14 — Файл logger.cpp

```
1. #include "logger.h"  
2.
```

```

3.  Logger::Logger(const QString& filename) : logFile(filename) {
4.      if (!logFile.open(QIODevice::WriteOnly | QIODevice::Append |
        QIODevice::Text)) {
5.          qWarning() << "Could not open log file:" << filename;
6.      }
7.      logStream.setDevice(&logFile);
8.  }
9.
10. void Logger::log(Severity severity, const QString& message) {
11.     QString severityString;
12.     switch (severity) {
13.         case Debug:    severityString = "Debug";    break;
14.         case Info:     severityString = "Info";     break;
15.         case Warning:  severityString = "Warning";  break;
16.         case Error:    severityString = "Error";    break;
17.         case Fatal:    severityString = "Fatal";    break;
18.     }
19.
20.     logStream << QDateTime::currentDateTime().toString("yyyy-MM-dd
        hh:mm:ss.zzz ")
21.         << "[" << severityString << "]" "
22.         << message << Qt::endl;
23.     logStream.flush();
24. }
25.
26. Logger::~~Logger() {
27.     logFile.close();
28. }

```

<Листинги оформляются созданным стилем «Оформление кода» и «Нумерация строк кода». Изменять эти стили (это категория «Стили пользователя») - не разрешается!>

<Подсветка (колоризация) синтаксиса не допускается!>

<Параметры кода: Шрифт - Source Code Pro (точно! не Source Code Pro Semibold), Кегль — 11 pt>

ПРИЛОЖЕНИЕ Б

ТЕСТ-СЬЮТ 1: МОДУЛЬ ПРОДАЖ

ID: TS-SALE-001

Название: Модульное тестирование модуля продаж

Цель: Проверка корректности оформления продажи: создание записи, сохранение суммы и способа оплаты, добавление позиций чека, списание остатков со склада, контроль ограничений по количеству товара

Проверяемые требования: REQ-003, FUNC-010, FUNC-011, FUNC-012, FUNC-015, DATA-003, DATA-004

Проверяемый модуль: Модуль продаж

Конфигурация выполнения: Qt Test, SQLite In-Memory Database

Приоритет: Высокий

Исполнитель: Дроботов Максим

Среда тестирования: Qt Framework (6.x), C++17, SQLite In-Memory Database

ID	Приоритет	Название	Предусловия	Шаги	Ожидаемый результат	Фактический результат	Статус
TC_SALE_01	Высокий	Создание продажи и сохранение в БД	Инициализирована тестовая БД с таблицами sales, employees, payment_methods	1. Вызвать createSale(500.0, 1) 2. Выполнить SELECT COUNT(*) FROM sales	Запись создана, sale_id > 0, количество записей = 1	sale_id > 0, COUNT(*) = 1	Пройден успешно
TC_SALE_02	Высокий	Корректность сохранения суммы продажи	Создана продажа на сумму 1234.56 руб.	1. Вызвать createSale(1234.56, 1) 2. Запросить final_amount из таблицы sales	Значение final_amount = 1234.56	1234.56	Пройден успешно
TC_SALE_03	Высокий	Добавление позиции в sale_items	Созданы лекарство, складская позиция и продажа	1. Выполнить INSERT INTO sale_items 2. Проверить COUNT(*)	Позиция добавлена, количество = 1	COUNT(*) = 1	Пройден успешно

TC_SALE_04	Высокий	Списание остатка после продажи	На складе 20 ед. товара, оформлена продажа 3 ед.	1. Добавить позицию с quantity = 3 2. Выполнить UPDATE stock SET quantity = quantity — 3 3. Запросить остаток	Остаток уменьшился с 20 до 17	Остаток = 17	Пройден успешно
TC_SALE_05	Высокий	Контроль: нельзя продать больше остатка	На складе 5 ед. товара	1. Запросить available 2. Сравнить с requested = 10	available (5) < requested (10), продажа невозможна	5 < 10, условие выполнено	Пройден успешно
TC_SALE_06	Средний	Сохранение способа оплаты	Создана продажа с payment_method_id = 2 (Карта)	1. Вызвать createSale(300.0, 2) 2. Запросить payment_method_id	Значение = 2 (Карта)	2	Пройден успешно
TC_SALE_07	Средний	Несколько позиций в одной продаже	Созданы два лекарства и две складские	1. Добавить две позиции в sale_items	Количество позиций = 2	COUNT(*) = 2	Пройден успешно

			позиции	2. Проверить COUNT(*) WHERE sale_id = ?			
--	--	--	---------	--	--	--	--

ТЕСТ-СЬЮТ 2: БЕЗОПАСНОСТЬ И АУТЕНТИФИКАЦИЯ

ID: TS-SEC-001.

Название: Модульное тестирование механизмов безопасности и аутентификации.

Цель: Проверка корректности хеширования паролей, соблюдения ограничений уникальности, обработки успешных и неудачных попыток входа в систему.

Проверяемые требования: REQ-005, FUNC-001, FUNC-002, FUNC-003, FUNC-004, QUAL-008, QUAL-009.

Конфигурация выполнения: Qt Test, SQLite In-Memory Database.

Модуль: Модуль авторизации.

Исполнитель: Дроботов Максим.

Среда тестирования: Qt Framework (6.x), C++17, SQLite In-Memory Database.

ID	Приоритет	Название	Предусловия	Шаги	Ожидаемый результат	Фактический результат	Статус
TC_VALID_01	Высокий	Валидация формата хранения паролей	Создана тестовая среда с таблицей employees	1. Выполнить вставку тестового пользователя через addUser 2. Напрямую запросить password_hash из базы данных	Хеш-сумма не совпадает с исходным паролем («123»), имеет длину 64 символа и остаётся неизменной при повторном хешировании	Получен хеш длиной 64 символа, не равный исходному паролю. Повторное хеширование дало идентичный результат	Пройден успешно
TC_VALID_02	Высокий	Проверка ограничения уникальности логина	В БД добавлен пользователь с логином «login». Структура таблицы изменена для включения ограничений NOT NULL и	1. Выполнить повторную вставку записи с дублирующимся логином «login»	Система отклоняет транзакцию, метод exec() возвращает false, предотвращая создание дубликата	Метод exec() вернул false, дублирующая запись не создана	Пройден успешно

			UNIQUE				
ТС_AUTH_01	Высокий	Успешная аутентификация с валидными учётными данными	В БД добавлен пользователь «Admin» с паролем «123» и ролью «Администратор»	1. Вызвать метод <code>loginUser("admin", "123")</code> у экземпляра Database	Метод возвращает true, выходные параметры корректно заполняются именем («Admin») и ролью («Администратор»)	Метод вернул true, имя: «Admin», роль: «Администратор»	Пройден успешно
ТС_AUTH_02	Высокий	Обработка ошибки при неверном пароле	В БД добавлен пользователь «Admin» с паролем «123»	1. Вызвать метод <code>loginUser("admin", "wrong_password")</code>	Метод возвращает false, <code>getLastError()</code> содержит сообщение «Неверный пароль»	Метод вернул false, ошибка: «Неверный пароль»	Пройден успешно
ТС_AUTH_03	Средний	Обработка ошибки при	База данных инициализирована	1. Вызвать метод	Метод возвращает	Метод вернул false, ошибка:	Пройден успешно

		отсутствии пользователя в системе	на, таблицы пусты	loginUser("ghost ", "123")	false, getLastError() возвращает строку «Пользователь не найден»	«Пользователь не найден»	
--	--	---	----------------------	-------------------------------	---	-----------------------------	--

ТЕСТ-СЬЮТ 3: УПРАВЛЕНИЕ СЕССИЯМИ И ПРАВАМИ ДОСТУПА

ID TS-SESS-001.

Название Модульное тестирование управления сессиями и разграничения прав доступа.

Цель Проверка корректности установки и сброса пользовательской сессии, а также верификация логики разграничения ролей «Администратор» и «Фармацевт».

Проверяемые требования REQ-005, FUNC-003, QUAL-010, QUAL-011.

Конфигурация выполнения Qt Test, изолированный экземпляр UserSession.

Приоритет Средний.

Модуль Управление сессиями и правами.

Исполнитель Дроботов Максим.

Среда тестирования Qt Framework (6.x), C++17.

ID	Приоритет	Название	Предусловия	Шаги	Ожидаемый результат	Фактический результат	Статус
TC_SESS_01	Средний	Определение прав администратора в сессии	Объект UserSession очищен	1. Установить пользователя с ролью «Администратор» 2. Проверить права доступа	isAdmin() возвращает true, идентификатор и имя пользователя корректны	isAdmin() = true, getUserId() = 1, getUsername() = «Admin»	Пройден успешно
TC_SESS_02	Средний	Валидация сброса сессии и прав обычного пользователя	Объект UserSession очищен	1. Установить пользователя с ролью «Фармацевт» 2. Проверить права 3. Вызвать clear() 4. Проверить состояние	isAdmin() возвращает false. После сброса getUserId() возвращает -1, getUsername() пуст	До сброса: isAdmin() = false. После сброса: ID = -1, имя пусто, isAdmin() = false	Пройден успешно

ТЕСТ-СЬЮТ 4: CRUD-ОПЕРАЦИИ И ЦЕЛОСТНОСТЬ ДАННЫХ

ID: TS-DATA-001.

Название: Модульное тестирование CRUD-операций и ограничений целостности данных.

Цель: Проверка корректности выполнения операций создания, чтения, обновления и удаления сущностей (сотрудники), точности поиска, а также соблюдения ограничений внешних ключей.

Проверяемые требования: FUNC-005, FUNC-018, FUNC-030, FUNC-031, DATA-007, DATA-008.

Конфигурация выполнения: Qt Test, SQLite In-Memory Database.

Приоритет: Высокий.

Модуль: CRUD-операции и работа с данными.

Исполнитель: Дроботов Максим.

Среда тестирования: Qt Framework (6.x), C++17, SQLite In-Memory Database.

ID	Приоритет	Название	Предусловия	Шаги	Ожидаемый результат	Фактический результат	Статус
TC_CRUD_01	Высокий	Корректность добавления основной сущности (Сотрудник)	Создана структура БД	1. Выполнить добавление пользователя через addUser 2. Выполнить SELECT COUNT(*) FROM employees	Количество записей в таблице employees равно 1	COUNT(*) = 1, запись успешно добавлена	Пройден успешно
TC_CRUD_02	Высокий	Корректность обновления атрибутов сущности (Сотрудник)	В БД добавлен пользователь с логином «test»	1. Выполнить UPDATE для смены имени на «Updated» 2. Запросить имя по логину «test»	Метод exec() возвращает true, имя пользователя в БД изменяется на «Updated»	exec() = true, full_name = «Updated»	Пройден успешно
TC_CRUD_03	Средний	Корректность удаления сущности	В БД добавлен пользователь с	1. Выполнить DELETE для логина «test»	Количество записей в таблице	COUNT(*) = 0, запись успешно удалена	Пройден успешно

ID	Приоритет	Название	Предусловия	Шаги	Ожидаемый результат	Фактический результат	Статус
		(Сотрудник)	логином «test»	2. Пересчитать количество сотрудников	employees становится равно 0		
TC_SRCH_01	Средний	Точность поиска сущностей по ключевому слову	В БД добавлены «Иван Иванов» и «Пётр Петров»	1. Выполнить SELECT COUNT(*) WHERE full_name LIKE '%Иван%'	Результат запроса равен 1	COUNT(*) = 1, найден один пользователь	Пройден успешно
TC_SRCH_02	Низкий	Поиск при отсутствии совпадений	В БД добавлены пользователи	1. Выполнить запрос с фильтром %Нет%	Результат запроса равен 0	COUNT(*) = 0, совпадений не найдено	Пройден успешно

ПРИЛОЖЕНИЕ В

Документирование программного кода

<Созданы следующие стили:

- Блок научного руководителя
- Заголовок на титульной
- Листинг
- Основной текст на титульной
- Оформление кода
- Примечание
- Нумерация источников
- Нумерация строк в таблице
- Нумерация строк кода>

<Остальные измененные стили — это встроенные>